NOVARCH

CLIENT TECHNICAL REFERENCE

Complete Codebase Documentation

Public website / Backend API / Admin dashboard
Data models, workflows, security and deployment

DOCUMENT VERSION

1.1

SOURCE SNAPSHOT

21 October 2026

EVIDENCE BASIS

Working-tree source review

Prepared for client handover and developer maintenance

Contents

Clickable chapter and section links. API aliases are indexed in 5.2; per-handler contracts follow.

Contents . . . 2

1. Scope, evidence and project overview . . . 6

1.1 Client reference scope and reading conventions . . . 6

1.2 Business modules . . . 6

2. System architecture and execution flow . . . 7

2.1 Technology stack . . . 7

2.2 Runtime entry points . . . 8

2.3 Trust boundaries and communication . . . 8

2.4 Express request lifecycle . . . 8

3. Backend modules and business workflows . . . 10

3.1 Layer responsibilities . . . 10

3.2 Contact and service-lead lifecycle . . . 10

3.3 Community and Winter Arc lifecycle . . . 11

3.4 Checkout and payment reconciliation . . . 11

3.5 Dashboard metric semantics . . . 12

4. Request schemas and validation reference . . . 13

4.1 Structured request handling . . . 13

4.2 Contact public DTO . . . 13

4.3 Community public DTO . . . 14

4.4 Authentication, checkout and identifier rules . . . 14

5. Complete API reference . . . 15

5.1 Origins and successful API examples . . . 15

5.2 Endpoint registration index . . . 16

5.3 Health and authentication . . . 17

5.4 Contact inquiries and service leads . . . 19

5.5 Community and event applications . . . 22

5.6 Administrative metrics and payments . . . 27

5.7 Website-origin BFF and translation endpoints . . . 29

6. Public website routes and components . . . 33

6.1 Routing and page inventory . . . 33

6.2 Component composition . . . 34

6.3 Forms and visible state . . . 34

6.4 Shared state, styling and behavior . . . 34

6.5 Language switching and legal content . . . 35

7. Administrative dashboard . . . 36

7.1 Routes and access . . . 36

7.2 Data loading and API client . . . 36

7.3 Tables, details and feature mapping . . . 36

7.4 Dashboard workflow and session continuity . . . 37

8. Database schema and persistence . . . 38

8.1 Connection and collections . . . 38

8.2 Contact fields . . . 39

8.3 Community fields . . . 40

8.4 Projects and security records . . . 41

8.5 Data queries and lifecycle . . . 41

9. Authentication, authorization and security . . . 42

9.1 Admin session flow . . . 42

9.2 Implemented security controls . . . 43

9.3 Production configuration and release readiness . . . 43

10. External integrations 44

11. Environment configuration and deployment 45

11.1 Environment-variable reference 45

11.2 Local setup and verified commands 46

11.3 Provisioning and deployment architecture 47

11.4 Operations and service management 48

12. Testing and ongoing maintenance 49

12.1 Existing test inventory 49

12.2 Operational scripts 49

12.3 Maintenance and continuous development 50

13. Verification record and handover 51

14. File and module reference 52

frontend/src/app 53

frontend/src/components 55

frontend/src/context 56

frontend/src/data 57

frontend/src/hooks 57

frontend/src/lib 58

frontend/src/services 58

frontend/src/types 59

backend/src/config 59

backend/src/controllers 60

backend/src/data 60

backend/src/middleware 60

backend/src/models 61

backend/src/repositories . . . 61

backend/src/routes . . . 61

backend/src/scripts . . . 62

backend/src . . . 62

backend/src/services . . . 62

backend/src/types . . . 63

backend/src/utils . . . 63

dashboard/src . . . 63

dashboard/src/components . . . 64

dashboard/src/services . . . 64

dashboard/src/types . . . 64

Supporting configuration and assets . . . 65

1. Scope, evidence and project overview

NOVARCH is a public services website and lead-intake platform with a separate administrative command portal. It presents four service families, captures general inquiries and package leads, accepts community and Winter Arc event applications, and supports one-time Stripe purchases of fixed-price discovery packages. Administrators review records, change their status, delete them, and email Winter Arc applicants.

This reference documents the working tree on 21 October 2026, version 1.1 of this documentation. The reference captures the current project working tree for client handover. Package versions below come from lockfiles, with manifest ranges stated where useful. The accompanying build manifest records hashes used to verify preservation.

1.1 Client reference scope and reading conventions

- Verified implementation means a behavior is directly supported by the inspected source. Runtime acceptance testing belongs to the deployment process.
- Illustrative JSON examples use invented names, example.com addresses, synthetic identifiers and dummy token references. Optional fields may be omitted.
- Backend origin means the Express service. Website origin means the Next.js application.
- Application source inventory: 168 TS/TSX/CSS files and 23,238 lines at review time. Generated output, dependency trees and binary assets are excluded from source analysis. Configuration, manifests, lockfiles and operational scripts were inspected separately.
- Endpoint inventory: 21 Express handler contracts plus 5 Next.js handlers. Express aliases expand to 41 registrations; with Next.js this is 46 method/path registrations across the two origins.
- References use repository-relative paths so the document remains portable. The module inventory in Chapter 14 provides file-level navigation. Deployment-specific values, provider provisioning and runtime acceptance are handled in the target environment.

1.2 Business modules

Module	Business behavior	Persistence / external dependency
Website and service catalog	Marketing pages, founders, service descriptions, package selection	Local typed content modules
Contact intake	General messages and service package inquiries	Contact collection; transactional email
Community	Membership interest and profile capture	Community collection; acknowledgment and admin notification
Winter Arc	Curated event application, referral capture, separate consent choices	Community collection distinguished by applicationType
Fixed-price checkout	EUR one-time Blueprint purchases through hosted Checkout	Stripe session and Contact payment record
Command portal	Record lists, search, status changes, detail views, deletion and event replies	Direct authenticated Express requests
Legal information	Bilingual event hub and dedicated legal information pages	Structured bilingual information
HackTheFlag	Challenge information, external link and QR presentation	Dedicated external challenge environment

2. System architecture and execution flow

2.1 Technology stack

Layer	Verified technology / version	Role
Public website	Next.js 16.4.2; React 19.2.8; TypeScript 5.9.3 locked	App Router pages, server layout, client interactivity, same-origin form handlers
Website presentation	Tailwind CSS 4; Framer Motion 13.1.0 manifest range; Lucide React	Theme tokens, responsive layouts, reveal and service animations
Backend	Node.js; Express 4.22.3 locked; TypeScript 5.9.3 locked	REST handlers, security middleware and business services
Data	Mongoose 9.9.3 locked; MongoDB	Documents, indexes, revocable sessions, shared rate counters
Dashboard	React 19.2.8; Vite 6.4.3; React Router DOM 7.18.2 manifest range	Client-side administrative SPA
Payments	Stripe server SDK 17.7.0; @stripe/stripe-js 5.6.0 manifest range	Server-generated hosted Checkout; server SDK performs actual payment operations
Email	Nodemailer 10.1.1 manifest range; Resend HTTP API	Transactional acknowledgment, admin notification and event replies
Validation/security	Zod 3.24.2 range; bcryptjs 3.0.3 range; jsonwebtoken 9.0.3 range; Helmet 8.3.0 range	DTO validation, credential verification, JWT validation, API headers
Tooling	npm lockfile v3; tsx; ESLint 9; PostCSS; Tailwind Vite plugin	Installation, development execution, compilation and linting

Node 20.19+ within the 20.x line or a compatible 22+ release satisfies these three declared ranges.

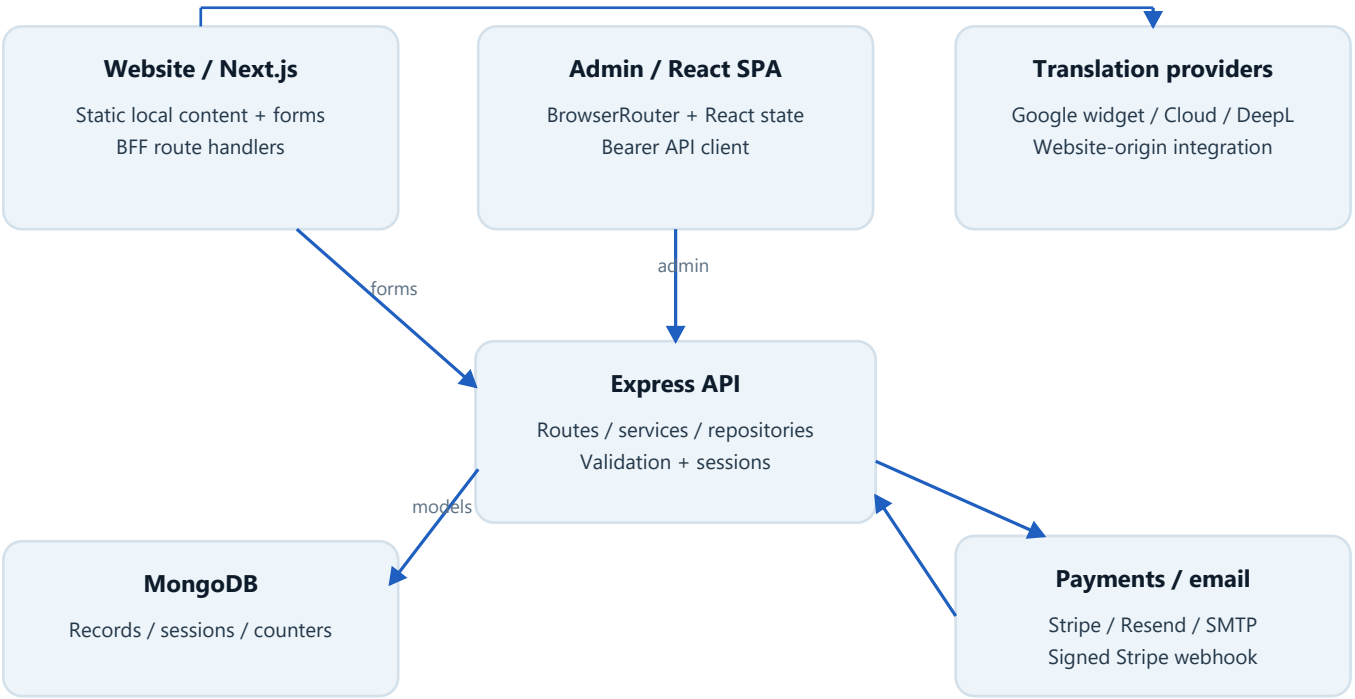


Figure 1. Implemented applications and external boundaries. Local website content is separate from MongoDB project records.

2.2 Runtime entry points

Entry	Execution
frontend/src/app/layout.tsx	Loads global styles and Inter font, applies dark class, wraps children with ThemeProvider, Navbar, ScrollToTop and Footer; main has top clearance for fixed navigation
frontend/src/app/page.tsx	Composes Hero, Services, Process, WhyNovarch, BuildWithUs and CTA sections
frontend/src/app/api/*/route.ts	Next.js server endpoints for signed form tokens, form forwarding and private translation API
backend/src/server.ts	Configures Express middleware, mounts route aliases, connects MongoDB, calls the no-op contact seed method, then listens outside test/Vercel
dashboard/src/main.tsx	Mounts the React App in the HTML root; App owns authentication gate, shared data and BrowserRouter
backend/vercel.json	Sends all deployment paths to src/server.ts using @vercel/node; exported app is the serverless entry

2.3 Trust boundaries and communication

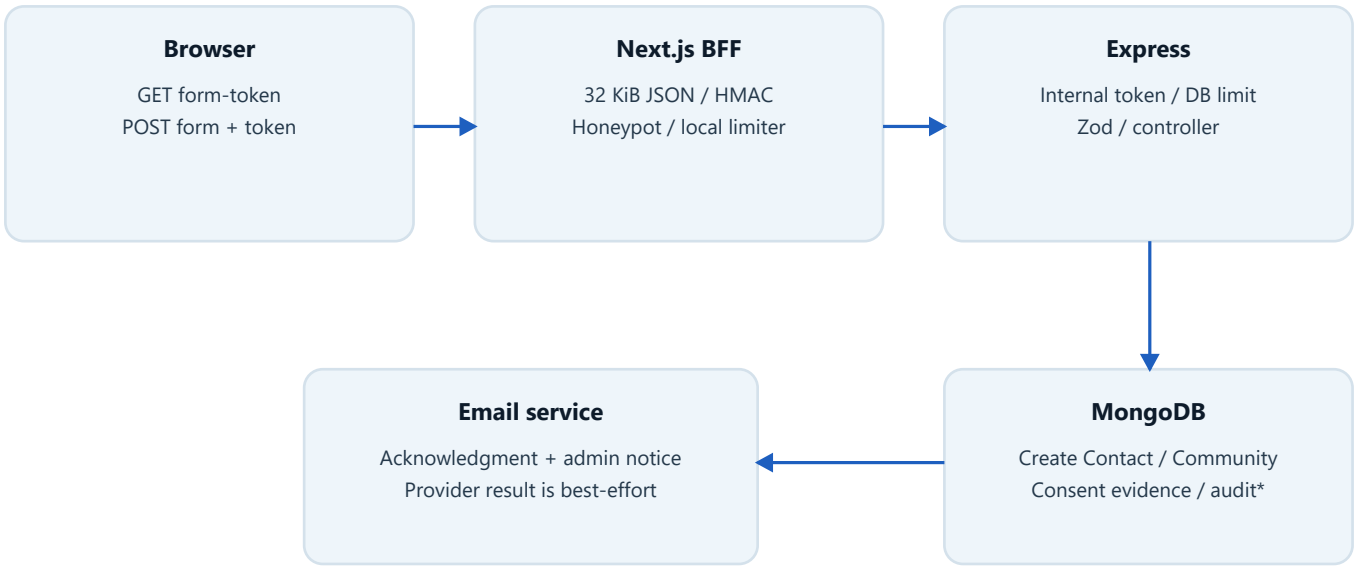
Public forms send JSON to the website origin. Next.js validates the request boundary and signed form token, then forwards a selected payload to Express with an optional shared internal token. The dashboard bypasses this forwarding layer and talks directly to Express with a Bearer token. Browser checkout creation and payment verification also talk directly to Express. Stripe sends signed webhooks directly to the backend; those requests require a raw JSON body.

MongoDB holds all operational records. The backend also contains a reusable Project service/model/repository layer. Active protected endpoints use validated admin sessions; the codebase also defines reusable role-permission helpers.

2.4 Express request lifecycle

The server disables X-Powered-By, validates TRUST_PROXY_HOPS (integer 0-5), selects the simple query parser, installs Helmet and compression, and applies a Mongo-backed general limiter. CORS is then evaluated. Raw body parsers are installed for both webhook aliases before general JSON/urlencoded parsers (1 MB). Login and checkout creation get additional sensitive-operation limiters. A separate per-account login limiter follows body parsing. Routers dispatch to controllers/services/repositories; errors reaching next(error) are normalized by errorHandler.

The shared MongoDB-backed limiter supports consistent request controls. The health handler reports database connection state and process uptime for operational visibility.



* Contact workflows include explicit audit recording.

Figure 2. Signed form forwarding, persistent intake and best-effort transactional notifications.

3. Backend modules and business workflows

3.1 Layer responsibilities

Directory / module	Responsibility and interactions
config/index.ts	Loads dotenv; exposes runtime, database, Stripe, URL, JWT, admin and internal-token configuration; rejects unsafe production auth/database configuration
config/database.ts	Mongoose connection/pool management and connection-state reporting; failure throws status 503
config/packageCatalog.ts	Canonical package IDs, display names, EUR prices and purchasability; case-insensitive ID/slug/name lookup
config/smtp.ts	Decrypts encrypted SMTP credentials; builds transport settings or returns null
routes	Registered method/path definitions; checkout implementation lives in its router rather than a controller
controllers	Parse DTOs, reject malformed identifiers, select workflow, map service results to HTTP
services/contact.service.ts	Contact persistence orchestration, audit events, two best-effort transactional email deliveries
services/community.service.ts	Community/event creation, list/detail, status, deletion and marketing withdrawal; returns acknowledgment email status
services/dashboard.service.ts	Computes contact-derived metrics in application memory after loading all contacts
services/session.service.ts	Pinned JWT validation plus live database session lookup and credential-version binding
services/email.service.ts	Provider selection, HTML/text templates, transactional replies and marketing eligibility helper
services/audit.service.ts	Persists SecurityEvent and keeps newest 500 entries in process memory; recent-log helper returns memory only
repositories	Mongoose reads/writes and document normalization; retain legacy fallback code that is not reached on connection exceptions
models	Six operational model families including the inline SecurityCounter model in middleware
types	DTO schemas and TypeScript shapes for contacts, community/events, projects and dashboard metrics
utils	Resource-ID format guard, timing comparison, AES-GCM secret helpers and escaped email-template views

3.2 Contact and service-lead lifecycle

The contact controller parses CreatePublicContactSchema. If planName, preferredService or serviceType matches a catalog entry, the controller overwrites plan name/tier/price/value/service values from the server catalog. A submission is a service lead only when submissionType equals service_lead; otherwise it is a message.

The repository derives source from submission type, builds serviceType from the selected performances when needed, and chooses message from message, news or a server default. Estimated value comes from canonical plan price when available, otherwise a category value map, otherwise zero for a general message or 490 for a service lead. The category map supports current and legacy service categories for pipeline estimation. It derives pipeline estimates from canonical package values and selected service categories.

After creation, ContactService awaits an audit write, then awaits Promise.allSettled for admin notification and submitter acknowledgment. Status changes and deletions are also audited. Record creation, audit recording and email delivery are separate workflow stages.

3.3 Community and Winter Arc lifecycle

The controller selects Winter Arc when applicationType is winter-arc or any of the three application-answer fields is truthy. It then validates the selected schema. Standard community submission requires fullName and email; Winter Arc additionally requires city, organization and all three substantive answers. The resulting record shares the Community model and uses applicationType to distinguish the workflows.

The repository sets source from the explicit source, UTM source or Website Form. It records marketing and photo choices separately, defaults them false, and creates consent timestamps/version v1-2026-10 only when true. Registration succeeds with either consent false. The service sends an acknowledgment and admin notification concurrently and returns the acknowledgment result as emailStatus.

Marketing withdrawal sets marketingConsent false and marketingConsentWithdrawnAt to server time, retaining original consent evidence. The canSendMarketingEmail helper checks active consent and withdrawal evidence. The protected withdrawal endpoint supports administrative handling, while photography preferences follow the event contact process.

3.4 Checkout and payment reconciliation

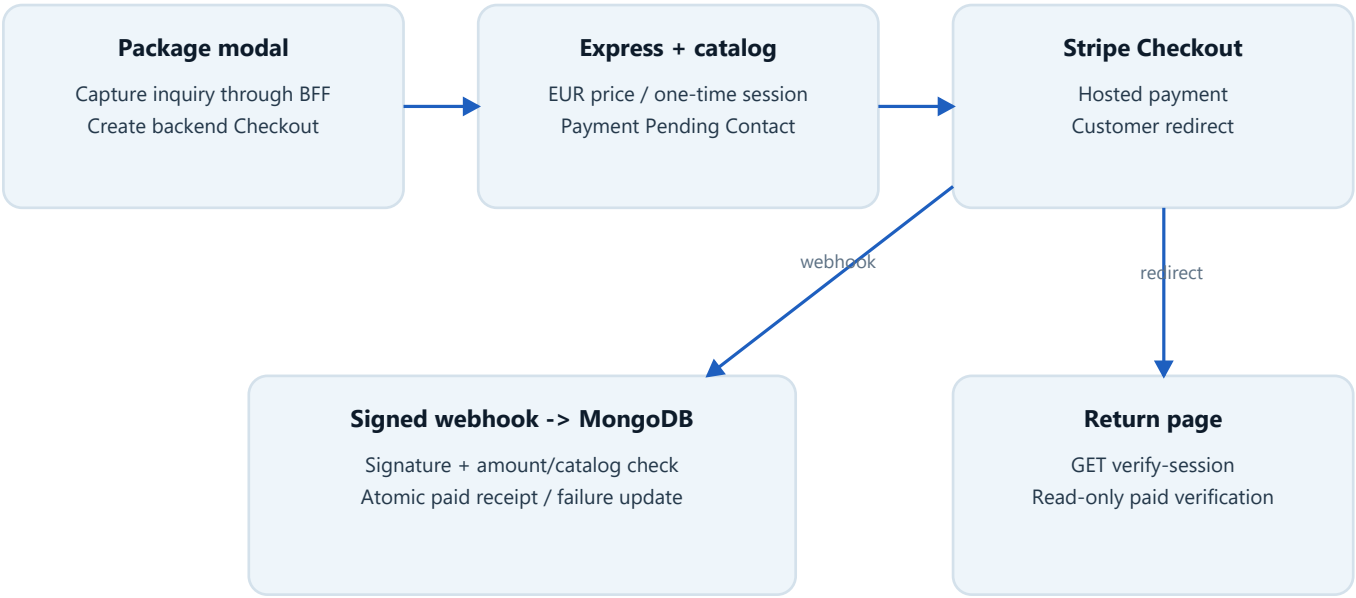


Figure 3. Hosted checkout and signed webhook reconciliation are distinct from return-page verification.

Only the four catalog entries below are directly purchasable. Price, currency and quantity are server-owned. The frontend passes a package name; the backend also accepts packageId or serviceSlug with precedence packageId, then serviceSlug, then packageName.

Package ID	Service / package	EUR	Unit amount (cents)
digital-launch-basic	Digital Launch / Launch Blueprint	490	49000
automation-integration-basic	Automation & Integration / Workflow Blueprint	690	69000
ai-workflow-basic	AI Workflow / AI Evidence Sprint	950	95000

Package ID	Service / package	EUR	Unit amount (cents)
custom-software-basic	Custom Software / Solution Blueprint	1500	150000

The modal first records a service lead through the website BFF. Premium or from-priced implementation requests finish with a qualification confirmation. Basic requests then create a Stripe session directly through Express. That backend call creates a second Contact record with Payment Pending and a unique session ID. The workflow records inquiry interest separately from the payment-linked Checkout record.

Stripe session creation uses mode payment, card payment method, quantity 1, EUR and metadata containing selected package and submitter details. Success redirects to FRONTEND_URL/payment-success?session_id=... and cancellation to /services. If persistence of the pending lead fails, the handler tries to expire the Stripe session, then returns 500.

The webhook verifies Stripe's signature before any database work. Completed or async-success sessions must be paid, use EUR, and match the canonical catalog amount. Sparse unique session/payment-intent indexes plus duplicate-key handling make successful receipt retries idempotent when indexes exist. A verified duplicate receipt returns already_processed. Email notification follows successful receipt persistence.

Unrecognized signed event types are acknowledged after database connection.

3.5 Dashboard metric semantics

DashboardService loads all contacts newest-first and counts total, New, Contacted, Proposal Sent and Closed. Conversion rate is rounded Closed / total * 100. Estimated pipeline sums estimatedValue for every status except Closed, including Paid and Payment Declined. The metrics response includes activeAutomatedWorkflows and a recent-activity list of the five newest contact creations labeled new_lead.

4. Request schemas and validation reference

4.1 Structured request handling

Express JSON and urlencoded parsers accept up to 1 MB. Website readObject requires application/json, streams at most 32,768 bytes before parsing, requires a non-null object, and permits at most 40 top-level keys. Consent fields must be booleans. performances/interests are string arrays with at most 15 elements of 100 characters each. referral objects allow at most five string values of up to 500 characters. estimatedValue must be a finite number; other supplied non-null fields must be strings of at most 3,000 characters. Endpoint-specific DTOs impose narrower rules after forwarding.

Website forms require form_token and an empty/absent website_hp honeypot. BFF rate limiting is process-local; backend counters are shared in MongoDB.

4.2 Contact public DTO

Source: backend/src/types/contact.types.ts, CreatePublicContactSchema. Required fields are name and email; the remaining fields are optional. General Zod object parsing strips unknown keys rather than rejecting them.

Field	Type / validation	Meaning
name	string, 2-100 characters	Submitter name; schema does not trim before minimum-length check, model trims on persistence
email	valid email string, lowercased	Reply address; this schema does not impose an explicit maximum
phone / countryCode / company	strings max 50 / 10 / 100	Optional contact context
submissionType	service_lead or message	Defaults in repository to message when not service_lead
serviceType / preferredService	strings max 500 / 200	Service context; catalog lookup may overwrite
planName / planTier	string max 100; Basic or Premium	Selected package context; canonical catalog overrides on match
planPrice / estimatedValue	string max 100; number	Accepted by schema but overwritten or deleted by public controller
performances	array max 15; each string max 100	Service/topic selections used for source-side valuation
news / message	strings max 3000	Narrative content; message takes precedence in repository
website_hp	optional string of length zero	Direct API honeypot check

Contact status updates accept exactly New, Contacted, Proposal Sent, Closed, Paid, Payment Declined or Payment Pending.

4.3 Community public DTO

Field	Standard community	Winter Arc
applicationType	optional community literal	winter-arc literal; default winter-arc
fullName	trimmed string 2-100, required	same
email	trimmed valid email, lowercased, required	same
city	optional trimmed string max 100	required trimmed string 1-100
organization	optional trimmed string max 150	required trimmed string 1-150
country / role	optional trimmed strings max 100 / 150	community-profile fields
contactNumber / countryCode	optional trimmed strings max 50 / 10	same
whatYouDo / whatCanYouOffer / whatAreYouLookingFor	event-specific fields	each required trimmed string 2-2000
linkedinUrl / portfolioUrl	event-specific fields	optional trimmed string max 500; empty or HTTP(S), no URL credentials
proposedGuest	event-specific field	optional trimmed string max 1000
interests / message	optional max 10 strings, each max 100; message max 3000	not Winter Arc schema fields
referral	string max 200 or referral object	referral object only
source	optional string max 100	same; client source label is permitted
marketingConsent / photoConsent	optional booleans; default false	same
website_hp	optional zero-length string	same

Referral object: optional utmSource, utmMedium, utmCampaign and utmContent strings, each max 150; optional referrer max 500. Reply body: trimmed subject 1-200 and message 1-10000 characters.

4.4 Authentication, checkout and identifier rules

Login is a strict Zod object with only email and password. Email is trimmed, valid, lowercased and max 254. Password is 1-72 characters and at most 72 UTF-8 bytes; the setup hashing utility requires 12-72 bytes.

Checkout: userName trimmed 2-100 and userEmail trimmed valid email/lowercase are required. serviceSlug, packageId and packageName are optional trimmed strings max 200; one must resolve to a purchasable catalog item. userPhone max 50, userCompany max 100 and userNotes max 500 are optional trimmed strings. Extra price/user fields are stripped. The checkout request applies its own concise notes validation.

5. Complete API reference

5.1 Origins and successful API examples

Canonical paths below use /api on Express. Each contract lists every mounted alias. Website-origin endpoints are documented separately in 5.7. All JSON examples in this chapter are illustrative source-derived shapes; token values, IDs, timestamps, Stripe IDs and provider responses are synthetic.

Symbol	Meaning
Backend origin	Configured Express service, for example http://localhost:5000 in local development
Website origin	Next.js service, normally http://localhost:3000 in local development
ADMIN	Authorization: Bearer <admin-session-jwt>; validated claims plus live session
INTERNAL	X-Internal-Token: <server-shared-secret> when INTERNAL_API_SECRET is set; server-to-server only
JSON	Content-Type: application/json for request bodies; JSON response unless specified

5.2 Endpoint registration index

Origin	Method	All registered paths	Handler source
Backend	POST	/api/auth/login	backend/src/routes/auth.routes.ts
Backend	GET	/api/auth/verify	backend/src/routes/auth.routes.ts
Backend	POST	/api/auth/logout	backend/src/routes/auth.routes.ts
Backend	POST	/checkout/create-session; /api/checkout/create-session	backend/src/routes/checkout.routes.ts
Backend	GET	/checkout/verify-session; /api/checkout/verify-session	backend/src/routes/checkout.routes.ts
Backend	POST	/checkout/webhook; /api/checkout/webhook	backend/src/routes/checkout.routes.ts
Backend	POST	/api/community	backend/src/routes/community.routes.ts
Backend	GET	/api/community	backend/src/routes/community.routes.ts
Backend	POST	/api/community/:id/reply	backend/src/routes/community.routes.ts
Backend	GET	/api/community/:id	backend/src/routes/community.routes.ts
Backend	PATCH	/api/community/:id/status	backend/src/routes/community.routes.ts
Backend	PATCH	/api/community/:id/withdraw-marketing-consent	backend/src/routes/community.routes.ts
Backend	DELETE	/api/community/:id	backend/src/routes/community.routes.ts
Backend	POST	/contact; /contacts; /api/contact; /api/contacts	backend/src/routes/contact.routes.ts
Backend	GET	/contact; /contacts; /api/contact; /api/contacts	backend/src/routes/contact.routes.ts
Backend	GET	/contact/:id; /contacts/:id; /api/contact/:id; /api/contacts/:id	backend/src/routes/contact.routes.ts
Backend	PATCH	/contact/:id/status; /contacts/:id/status; /api/contact/:id/status; /api/contacts/:id/status	backend/src/routes/contact.routes.ts
Backend	DELETE	/contact/:id; /contacts/:id; /api/contact/:id; /api/contacts/:id	backend/src/routes/contact.routes.ts
Backend	GET	/dashboard/stats; /api/dashboard/stats	backend/src/routes/dashboard.routes.ts
Backend	GET	/health; /api/health	backend/src/routes/health.routes.ts
Backend	GET	/	backend/src/server.ts
Website	POST	/api/community	frontend/src/app/api/community/route.ts
Website	POST	/api/contact	frontend/src/app/api/contact/route.ts
Website	GET	/api/form-token	frontend/src/app/api/form-token/route.ts
Website	POST	/api/translate	frontend/src/app/api/translate/route.ts
Website	POST	/api/winter-arc	frontend/src/app/api/winter-arc/route.ts

5.3 Health and authentication

GET / - backend service information

Contract	Verified behavior
Origin / aliases	Backend only; /
Handler	backend/src/server.ts, app.get
Headers / access	Public; no request content type required; R-general
Parameters / schema	No path/query fields

Returns fixed service identification.

Illustrative request (use the origin listed above):

```
GET / HTTP/1.1
```

Illustrative successful response:

```
{
  "name": "NOVARCH API Backend",
  "status": "online",
  "version": "1.0.0"
}
```

GET /api/health

Contract	Verified behavior
Origin / aliases	Backend: /health; /api/health
Handler	healthController.getHealth
Headers / access	Public; R-general
Parameters / schema	No path/query fields

Reads mongoose.connection.readyState and process-local start time.

Illustrative request (use the origin listed above):

```
GET /api/health HTTP/1.1
```

Illustrative successful response:

```
{
  "success": true,
  "status": "healthy",
  "database": "connected",
  "uptime": 120
}
```

POST /api/auth/login

Contract	Verified behavior
Origin / aliases	Backend: /api/auth/login
Handler	authController.login; session.service.ts
Headers / access	JSON, public; R-general + R-sensitive + R-account; response Cache-Control no-store
Parameters / schema	Strict Login DTO (Chapter 4); unknown keys rejected

The example email must be replaced with the configured ADMIN_USERNAME. bcrypt compare and timing-safe username check; audit AUTH_LOGIN even for rejected credentials after schema/config checks. On success connect DB, create random jti, sign HS256 claims and persist AdminSession.

Illustrative request (use the origin listed above):

```
POST /api/auth/login HTTP/1.1
Content-Type: application/json

{
  "email": "admin@example.com",
  "password": "&lt;configured-admin-password&gt;"
}
```

Illustrative successful response:

```
{
  "success": true,
  "token": "&lt;two-hour-admin-jwt&gt;",
  "user": {
    "email": "admin@example.com",
    "role": "admin"
  }
}
```

GET /api/auth/verify

Contract	Verified behavior
Origin / aliases	Backend: /api/auth/verify
Handler	authController.verify; validateSession
Headers / access	ADMIN; response Cache-Control no-store; R-general
Parameters / schema	No path/query fields

Checks database session, expiry and credentialVersion. Claim timestamps in the example are illustrative.

Illustrative request (use the origin listed above):

```
GET /api/auth/verify HTTP/1.1
Authorization: Bearer &lt;admin-session-jwt&gt;
```

Illustrative successful response:

```
{
  "valid": true,
  "user": {
    "username": "admin@example.com",
    "role": "admin",
    "iat": 1792573200,
    "exp": 1792580400,
    "aud": "novarch-admin",
    "iss": "novarch-api",
    "jti": "&lt;session-uuid&gt;"
  }
}
```

POST /api/auth/logout

Contract	Verified behavior
Origin / aliases	Backend: /api/auth/logout
Handler	authController.logout
Headers / access	ADMIN; R-general
Parameters / schema	No request fields

Validates live session, deletes AdminSession by jti, then audits AUTH_LOGOUT. Browser storage clearing belongs to the dashboard.

Illustrative request (use the origin listed above):

```
POST /api/auth/logout HTTP/1.1
Authorization: Bearer &lt;admin-session-jwt&gt;
```

Illustrative successful response:

```
{
  "success": true
}
```

5.4 Contact inquiries and service leads

POST /api/contact

Contract	Verified behavior
Origin / aliases	Backend aliases: /contact; /contacts; /api/contact; /api/contacts
Handler	contactController.createContact
Headers / access	JSON; INTERNAL when configured; R-form plus R-general unless authenticated canonical BFF skip
Parameters / schema	Contact public DTO (4.2); no query/path fields

Catalog-normalizes selected package and overwrites/removes public valuation. ContactService -> ContactRepository.create -> Contact model, then AuditService and two transactional emails.

Illustrative request (use the origin listed above):

```
POST /api/contact HTTP/1.1
Content-Type: application/json
X-Internal-Token: &lt;shared-secret-if-configured&gt;

{
  "name": "Alex Morgan",
  "email": "alex@example.com",
  "submissionType": "message",
  "message": "Please discuss our workflow.",
  "performances": []
}
```

Illustrative successful response:

```
{
  "success": true,
  "message": "Inquiry received successfully. Our architecture team will be in
    touch.",
  "data": {
    "id": "507f1f77bcf86cd799439011"
  }
}
```

GET /api/contacts

Contract	Verified behavior
Origin / aliases	Backend aliases: /contact; /contacts; /api/contact; /api/contacts
Handler	contactController.getContacts
Headers / access	ADMIN; R-general
Parameters / schema	No supported list query fields; no pagination

ContactService.getAllInquiries -> repository.findAll -> Contact.find().sort(createdAt descending).lean(). Returns full records including payment references when present. Count equals returned array length.

Illustrative request (use the origin listed above):

```
GET /api/contacts HTTP/1.1
Authorization: Bearer &lt;admin-session-jwt&gt;
```

Illustrative successful response:

```
{
  "success": true,
  "count": 1,
  "data": [
    {
      "id": "507f1f77bcf86cd799439011",
      "submissionType": "message",
      "name": "Alex Morgan",
      "email": "alex@example.com",
      "performances": [],
      "serviceType": "General Inquiry",
      "preferredService": "General Inquiry",
      "message": "Please discuss our workflow.",
      "status": "New",
      "estimatedValue": 0,
      "source": "Website Form",
      "createdAt": "2026-10-21T09:00:00.000Z",
      "updatedAt": "2026-10-21T09:00:00.000Z"
    }
  ]
}
```

GET /api/contacts/:id

Contract	Verified behavior
Origin / aliases	Backend aliases: /contact/:id; /contacts/:id; /api/contact/:id; /api/contacts/:id
Handler	contactController.getContactById
Headers / access	ADMIN; R-general
Parameters / schema	id: resource pattern in 4.4; normally Mongo ObjectId; no query

ContactService.getInquiryById -> repository.findById. Unknown well-formed Mongo ID produces 404; allowed non-ObjectId values may cause cast/503.

Illustrative request (use the origin listed above):

```
GET /api/contacts/507f1f77bcf86cd799439011 HTTP/1.1
Authorization: Bearer &amp;lt;admin-session-jwt&amp;gt;
```

Illustrative successful response:

```
{
  "success": true,
  "data": {
    "id": "507f1f77bcf86cd799439011",
    "submissionType": "message",
    "name": "Alex Morgan",
    "email": "alex@example.com",
    "performances": [],
    "serviceType": "General Inquiry",
    "preferredService": "General Inquiry",
    "message": "Please discuss our workflow.",
    "status": "New",
    "estimatedValue": 0,
    "source": "Website Form",
    "createdAt": "2026-10-21T09:00:00.000Z",
    "updatedAt": "2026-10-21T09:00:00.000Z"
  }
}
```

PATCH /api/contacts/:id/status

Contract	Verified behavior
Origin / aliases	Backend aliases: /contact/:id/status; /contacts/:id/status; /api/contact/:id/status; /api/contacts/:id/status
Handler	contactController.updateContactStatus
Headers / access	JSON + ADMIN; R-general
Parameters / schema	id required; body status: Contact status enum (4.2)

Updates Contact status with validators and new document, then audits UPDATE_STATUS. Any enum transition is allowed, including setting payment labels manually.

Illustrative request (use the origin listed above):

```
PATCH /api/contacts/507f1f77bcf86cd799439011/status HTTP/1.1
Content-Type: application/json
Authorization: Bearer &amp;lt;admin-session-jwt&amp;gt;

{
  "status": "Contacted"
}
```

Illustrative successful response:

```
{
  "success": true,
  "message": "Status updated to Contacted",
  "data": {
    "id": "507f1f77bcf86cd799439011",
    "submissionType": "message",
    "name": "Alex Morgan",
    "email": "alex@example.com",
    "performances": [],
    "serviceType": "General Inquiry",
    "preferredService": "General Inquiry",
    "message": "Please discuss our workflow.",
    "status": "Contacted",
    "estimatedValue": 0,
    "source": "Website Form",
    "createdAt": "2026-10-21T09:00:00.000Z",
    "updatedAt": "2026-10-21T10:00:00.000Z"
  }
}
```


DELETE /api/contacts/:id

Contract	Verified behavior
Origin / aliases	Backend aliases: /contact/:id; /contacts/:id; /api/contact/:id; /api/contacts/:id
Handler	contactController.deleteContact
Headers / access	ADMIN; R-general
Parameters / schema	id required; no query

Illustrative request (use the origin listed above):

```
DELETE /api/contacts/507f1f77bcf86cd799439011 HTTP/1.1
Authorization: Bearer &lt;admin-session-jwt&gt;
```

Illustrative successful response:


```
{
  "success": true,
  "message": "Inquiry deleted successfully"
}
```


5.5 Community and event applications

POST /api/community

Contract	Verified behavior
Origin / aliases	Backend: /api/community
Handler	communityController.createMember
Headers / access	JSON + INTERNAL when configured; R-form plus R-general unless canonical BFF skip
Parameters / schema	Community or Winter Arc DTO (4.3), selected by discriminator/answer presence

Creates Community record with server-owned consent timestamps/default New. Standard community returns message "Community application received successfully. Welcome to NOVARCH!".

Illustrative request (use the origin listed above):

```
POST /api/community HTTP/1.1
Content-Type: application/json
X-Internal-Token: &lt;shared-secret-if-configured&gt;

{
  "applicationType": "winter-arc",
  "fullName": "Alex Morgan",
  "email": "alex@example.com",
  "city": "Ilmenau",
  "organization": "Example Studio",
  "whatYouDo": "I build workflow tools.",
  "whatCanYouOffer": "Product engineering experience.",
  "whatAreYouLookingFor": "Design collaborators.",
  "marketingConsent": false,
  "photoConsent": false
}
```

Illustrative successful response:

```
{
  "success": true,
  "message": "Thank you for applying to Winter Arc. Your application has been
    received. We'll review it and contact you if you're selected. Thanks for your
    interest in Treffen 01.",
  "data": {
    "id": "507f1f77bcf86cd799439011"
  },
  "emailStatus": "sent"
}
```

GET /api/community

Contract	Verified behavior
Origin / aliases	Backend: /api/community
Handler	communityController.getMembers
Headers / access	ADMIN; R-general
Parameters / schema	Optional query applicationType=community or winter-arc; unrecognized value ignored

CommunityService.getAllMembers -> repository.findAll(filter) -> Community find/sort/lean.

Illustrative request (use the origin listed above):

```
GET /api/community HTTP/1.1
Authorization: Bearer &lt;admin-session-jwt&gt;
```

Illustrative successful response:

```
{
  "success": true,
  "count": 1,
  "data": [
    {
      "id": "507f1f77bcf86cd799439011",
      "applicationType": "community",
      "fullName": "Alex Morgan",
      "email": "alex@example.com",
      "interests": [],
      "status": "New",
      "source": "Website Form",
      "marketingConsent": false,
      "marketingConsentAt": null,
      "marketingConsentWithdrawnAt": null,
      "photoConsent": false,
      "photoConsentAt": null,
      "createdAt": "2026-10-21T09:00:00.000Z",
      "updatedAt": "2026-10-21T09:00:00.000Z"
    }
  ]
}
```


GET /api/community/:id

Contract	Verified behavior
Origin / aliases	Backend: /api/community/:id
Handler	communityController.getMemberById
Headers / access	ADMIN; R-general
Parameters / schema	id: shared resource guard; no query

Read one Community record. Dashboard community details currently search shared list memory instead of using this endpoint.

Illustrative request (use the origin listed above):

```
GET /api/community/507f1f77bcf86cd799439011 HTTP/1.1
Authorization: Bearer &lt;admin-session-jwt&gt;
```

Illustrative successful response:


```
{
  "success": true,
  "data": {
    "id": "507f1f77bcf86cd799439011",
    "applicationType": "community",
    "fullName": "Alex Morgan",
    "email": "alex@example.com",
    "interests": [],
    "status": "New",
    "source": "Website Form",
    "marketingConsent": false,
    "marketingConsentAt": null,
    "marketingConsentWithdrawnAt": null,
    "photoConsent": false,
    "photoConsentAt": null,
    "createdAt": "2026-10-21T09:00:00.000Z",
    "updatedAt": "2026-10-21T09:00:00.000Z"
  }
}
```

PATCH /api/community/:id/status

Contract	Verified behavior
Origin / aliases	Backend: /api/community/:id/status
Handler	communityController.updateMemberStatus
Headers / access	JSON + ADMIN; R-general
Parameters / schema	id plus normalized Community status enum (4.3)

CommunityService -> updateStatus with validators and returned updated document.

Illustrative request (use the origin listed above):

```
PATCH /api/community/507f1f77bcf86cd799439011/status HTTP/1.1
Content-Type: application/json
Authorization: Bearer &lt;admin-session-jwt&gt;

{
  "status": "Reviewing"
}
```

Illustrative successful response:

```
{
  "success": true,
  "message": "Status updated to Reviewing",
  "data": {
    "id": "507f1f77bcf86cd799439011",
    "applicationType": "community",
    "fullName": "Alex Morgan",
    "email": "alex@example.com",
    "interests": [],
    "status": "Reviewing",
    "source": "Website Form",
    "marketingConsent": false,
    "marketingConsentAt": null,
    "marketingConsentWithdrawnAt": null,
    "photoConsent": false,
    "photoConsentAt": null,
    "createdAt": "2026-10-21T09:00:00.000Z",
    "updatedAt": "2026-10-21T10:00:00.000Z"
  }
}
```

PATCH /api/community/:id/withdraw-marketing-consent

Contract	Verified behavior
Origin / aliases	Backend: /api/community/:id/withdraw-marketing-consent
Handler	communityController.withdrawMarketingConsent
Headers / access	ADMIN; R-general
Parameters / schema	id required; body not read

Sets consent false and withdrawal/updated timestamps to server time. Original marketingConsentAt/version retained.

Illustrative request (use the origin listed above):

```
PATCH /api/community/507f1f77bcf86cd799439011/withdraw-marketing-consent HTTP/1.1
Authorization: Bearer &lt;admin-session-jwt&gt;
```

Illustrative successful response:

```
{
  "success": true,
  "message": "Marketing consent withdrawn successfully",
  "data": {
    "id": "507f1f77bcf86cd799439011",
    "marketingConsent": false,
    "marketingConsentWithdrawnAt": "2026-10-21T10:00:00.000Z"
  }
}
```

POST /api/community/:id/reply

Contract	Verified behavior
Origin / aliases	Backend: /api/community/:id/reply
Handler	communityController.replyWinterArc; emailService.sendWinterArcReply
Headers / access	JSON + ADMIN; R-general
Parameters / schema	id; subject 1-200 and message 1-10000 trimmed characters

Loads member; requires applicationType winter-arc; uses stored email as recipient. Calls generic transactional email with subject/plain message.

Illustrative request (use the origin listed above):

```
POST /api/community/507f1f77bcf86cd799439011/reply HTTP/1.1
Content-Type: application/json
Authorization: Bearer &lt;admin-session-jwt&gt;

{
  "subject": "Winter Arc application update",
  "message": "Thank you for your application. We will share event details
    directly."
}
```

Illustrative successful response:

```
{
  "success": true,
  "message": "Reply sent"
}
```

DELETE /api/community/:id

Contract	Verified behavior
Origin / aliases	Backend: /api/community/:id
Handler	communityController.deleteMember
Headers / access	ADMIN; R-general
Parameters / schema	id required; no query

Hard-deletes Community record via repository.

Illustrative request (use the origin listed above):

```
DELETE /api/community/507f1f77bcf86cd799439011 HTTP/1.1
Authorization: Bearer &lt;admin-session-jwt&gt;
```

Illustrative successful response:

```
{
  "success": true,
  "message": "Record deleted successfully"
}
```

5.6 Administrative metrics and payments

GET /api/dashboard/stats

Contract	Verified behavior
Origin / aliases	Backend aliases: /dashboard/stats; /api/dashboard/stats
Handler	dashboardController.getStats; DashboardService.getStats
Headers / access	ADMIN; R-general
Parameters / schema	No path/query/body fields

Reads all Contact records and calculates metrics in memory; definitions in 3.5. With records, recentActivity items contain id act-<contact-id>, description, timestamp and type new_lead.

Illustrative request (use the origin listed above):

```
GET /api/dashboard/stats HTTP/1.1
Authorization: Bearer &lt;admin-session-jwt&gt;
```

Illustrative successful response:

```
{
  "success": true,
  "data": {
    "totalInquiries": 0,
    "newLeads": 0,
    "contactedLeads": 0,
    "proposalsSent": 0,
    "closedDeals": 0,
    "estimatedPipelineValue": 0,
    "activeAutomatedWorkflows": 0,
    "conversionRate": 0,
    "recentActivity": []
  }
}
```

POST /api/checkout/create-session

Contract	Verified behavior
Origin / aliases	Backend aliases: /checkout/create-session; /api/checkout/create-session
Handler	routes/checkout.routes.ts POST /create-session
Headers / access	JSON; public; R-general + R-sensitive; no form token/internal header required
Parameters / schema	Checkout DTO 4.4; one catalog identifier; no query

Stripe SDK sessions.create with server catalog, EUR/card/payment/quantity1. Persists Payment Pending Contact keyed by Stripe ID. This handler catches DB exceptions as 500 rather than the shared 503 envelope.

Illustrative request (use the origin listed above):

```
POST /api/checkout/create-session HTTP/1.1
Content-Type: application/json

{
  "packageId": "digital-launch-basic",
  "userName": "Alex Morgan",
  "userEmail": "alex@example.com",
  "userCompany": "Example Studio",
  "userNotes": "Discovery package for our website."
}
```

Illustrative successful response:

```
{
  "url": "https://checkout.stripe.com/&lt;illustrative-session-url&gt;",
  "sessionId": "cs_test_example"
}
```

GET /api/checkout/verify-session

Contract	Verified behavior
Origin / aliases	Backend aliases: /checkout/verify-session; /api/checkout/verify-session
Handler	routes/checkout.routes.ts GET /verify-session
Headers / access	Public; R-general
Parameters / schema	session_id: required query string, cs_ prefix and <=200 characters

Retrieves session from Stripe and checks payment_status paid; amount_total divided by 100, currency uppercased.

Illustrative request (use the origin listed above):

```
GET /api/checkout/verify-session?session_id=cs_test_example HTTP/1.1
```

Illustrative successful response:

```
{
  "verified": true,
  "amountTotal": 490,
  "currency": "EUR"
}
```

POST /api/checkout/webhook

Contract	Verified behavior
Origin / aliases	Backend aliases: /checkout/webhook; /api/checkout/webhook
Handler	routes/checkout.routes.ts POST /webhook
Headers / access	Content-Type application/json, raw bytes + stripe-signature; no admin token; bypasses R-general
Parameters / schema	Stripe signed event schema; event IDs/session metadata supplied by Stripe

Example body requires a real matching Stripe signature over exact raw bytes; fabricated signatures are never valid. Models/calls: ContactModel.findOneAndUpdate/updateOne/exists, catalog, Stripe session list, admin payment notification. See payment sequence and atomic guard in 3.4.

Illustrative request (use the origin listed above):

```
POST /api/checkout/webhook HTTP/1.1
Content-Type: application/json
stripe-signature: &lt;provider-signature-for-exact-raw-body&gt;

{
  "id": "evt_example",
  "type": "checkout.session.completed",
  "data": {
    "object": {
      "id": "cs_test_example",
      "payment_status": "paid",
      "currency": "eur",
      "amount_total": 49000,
      "customer_email": "alex@example.com",
      "metadata": {
        "packageId": "digital-launch-basic",
        "userName": "Alex Morgan"
      }
    }
  }
}
```

Illustrative successful response:

```
{
  "received": true
}
```

5.7 Website-origin BFF and translation endpoints

GET /api/form-token

Contract	Verified behavior
Origin / aliases	Website only: /api/form-token
Handler	frontend/src/app/api/form-token/route.ts; issueFormToken
Headers / access	Public; Cache-Control no-store
Parameters / schema	No body/query/path

HMAC-SHA256 timestamp token using FORM_SIGNING_SECRET (dev fallback only outside production). Token valid after 1 second and up to 2 hours; reusable.

Illustrative request (use the origin listed above):

```
GET /api/form-token HTTP/1.1
```

Illustrative successful response:

```
{
  "token": "&lt;base64url-issuedAt.signature&gt;"
}
```

POST /api/contact - website forwarding

Contract	Verified behavior
Origin / aliases	Website only: /api/contact
Handler	frontend/src/app/api/contact/route.ts
Headers / access	JSON; public; signed form_token plus empty website_hp; BFF-form. No browser INTERNAL header: forwarding adds it server-side.
Parameters / schema	Contact inputs plus form_token string; name/email truthy required before backend; no query

readObject -> guardPublicForm -> required checks -> payload whitelist -> POST BACKEND_API_URL/contact with 20s timeout and optional internal/IP headers. Combines country code/phone and derives message/service values. Service-lead success text is Package subscription received and synchronized with NOVARCH backend. Errors use {error:<message>}.

Illustrative request (use the origin listed above):

```
POST /api/contact HTTP/1.1
Content-Type: application/json

{
  "name": "Alex Morgan",
  "email": "alex@example.com",
  "message": "Please discuss our workflow.",
  "form_token": "&lt;valid-form-token&gt;",
  "website_hp": ""
}
```

Illustrative successful response:

```
{
  "success": true,
  "message": "Project message received and synchronized with NOVARCH backend.",
  "data": {
    "id": "507f1f77bcf86cd799439011"
  }
}
```

POST /api/community - website forwarding

Contract	Verified behavior
Origin / aliases	Website only: /api/community
Handler	frontend/src/app/api/community/route.ts
Headers / access	JSON; public; signed form_token plus empty website_hp; BFF-form. No browser INTERNAL header: forwarding adds it server-side.
Parameters / schema	Community/Winter fields plus form_token; required name/email and conditional Winter fields

Selects application type, performs truthy required checks and trims/coerces selected text fields. Builds explicit payload; only literal true sets each consent true. Forwards POST BACKEND_API_URL/community with timeout and server headers. Returns backend message, created id and acknowledgment emailStatus. Strict typed/schema validation remains in Express.

Illustrative request (use the origin listed above):

```
POST /api/community HTTP/1.1
Content-Type: application/json

{
  "applicationType": "community",
  "fullName": "Alex Morgan",
  "email": "alex@example.com",
  "interests": [
    "Engineering"
  ],
  "form_token": "&lt;valid-form-token&gt;",
  "website_hp": ""
}
```

Illustrative successful response:

```
{
  "success": true,
  "message": "Community application received successfully. Welcome to NOVARCH!",
  "data": {
    "id": "507f1f77bcf86cd799439011"
  },
  "emailStatus": "unconfigured"
}
```

POST /api/winter-arc - forwarding alias

Contract	Verified behavior
Origin / aliases	Website only: /api/winter-arc
Handler	frontend/src/app/api/winter-arc/route.ts calls community POST
Headers / access	JSON; public; signed form_token plus empty website_hp; BFF-form. No browser INTERNAL header: forwarding adds it server-side.
Parameters / schema	Exactly the /api/community BFF schema; this alias does not force applicationType

Delegates the original Request unchanged to communityPost. Without winter-arc discriminator/answer fields, a standard community submission is possible even on this alias. The current WinterArcContent UI calls /api/community instead.

Illustrative request (use the origin listed above):

```
POST /api/winter-arc HTTP/1.1
Content-Type: application/json

{
  "applicationType": "winter-arc",
  "fullName": "Alex Morgan",
  "email": "alex@example.com",
  "city": "Ilmenau",
  "organization": "Example Studio",
  "whatYouDo": "I build workflow tools.",
  "whatCanYouOffer": "Product engineering experience.",
  "whatAreYouLookingFor": "Design collaborators.",
  "marketingConsent": false,
  "photoConsent": false,
  "form_token": "&lt;valid-form-token&gt;",
  "website_hp": ""
}
```

Illustrative successful response:

```
{
  "success": true,
  "message": "Thank you for applying to Winter Arc. Your application has been
    received. We'll review it and contact you if you're selected. Thanks for your
    interest in Treffen 01.",
  "data": {
    "id": "507f1f77bcf86cd799439011"
  },
  "emailStatus": "sent"
}
```

POST /api/translate

Contract	Verified behavior
Origin / aliases	Website only: /api/translate
Handler	frontend/src/app/api/translate/route.ts
Headers / access	JSON; X-Translation-Token: <private-shared-secret>; separate from public widget
Parameters / schema	text nonempty array <=15 strings, each <=3000 (readObject restriction); target/source en or de, defaults de/en

Timing-safe secret comparison before body parse. DeepL preferred, then Google Cloud on a non-OK response; each has 15s timeout.

Illustrative request (use the origin listed above):

```
POST /api/translate HTTP/1.1
Content-Type: application/json
X-Translation-Token: &lt;private-shared-secret&gt;

{
  "text": [
    "Build systems you can own."
  ],
  "target": "de",
  "source": "en"
}
```

Illustrative successful response:

```
{
  "provider": "deepl",
  "translations": [
    "Entwickeln Sie Systeme, die Ihnen gehören."
  ]
}
```

6. Public website routes and components

6.1 Routing and page inventory

Route	Behavior	Main composition / API dependency
/about	Disabled	Calls notFound(); older page code is commented out.
/case-studies/[slug]	Redirect	All slug routes redirect to /services; service-oriented compatibility route.
/case-studies	Redirect	redirect to /services; service-oriented compatibility route.
/contact	Public form	Query-driven topic selection, useFormToken, POST website /api/contact; submitting/error/success states.
/datenschutz	Privacy information	PermanentLegalPage privacy prop; dedicated privacy-information page; metadata noindex.
/founders	Team / portfolio	Local founder narrative, journey, venture cards, operating steps, principles and cofounder.
/hacktheflag	External challenge portal	HackTheFlagContent and QR/external link; challenge/scoring itself is outside this repository.
/impressum	Legal information	PermanentLegalPage; dedicated operator-information page; metadata noindex.
/legal/nda	Permanent redirect	To /legal/winter-arc-2026#confidentiality.
/legal	Permanent redirect	permanentRedirect to /legal/winter-arc-2026.
/legal/photography-consent	Permanent redirect	To /legal/winter-arc-2026#photography.
/legal/winter-arc-2026	Bilingual legal hub	EventLegalHub with section anchors; organizer-managed bilingual information; bilingual source content.
/our-community	Membership form	Profile/interests/referral, token and POST website /api/community.
/	Home	Hero > Services > Process > WhyNovarch > BuildWithUs > CTA; links to services/contact; local content.
/payment-success	Stripe return	Suspense + session_id query; direct Express verify-session call; loading/verified/failure UI.
/pricing	Pricing	ServicesSection reused beneath pricing introduction; fixed Blueprint/scoped implementation copy.
/services/ai-automation	Redirect	redirect to /services/automation-integration.
/services/ai-workflow	Service detail	Local ai-workflow service; evidence sprint/implementation presentation and package modal.
/services/ai-workflows	Redirect	redirect to /services/ai-workflow.
/services/automation-integration	Service detail	Local automation-integration service into shared template; workflow graphic and scope.
/services/custom-software	Service detail	Local custom-software service; solution Blueprint/implementation information.
/services/digital-launch	Service detail	Local digital-launch service into ServicePageTemplate, scope and package modal.

Route	Behavior	Main composition / API dependency
/services	Service index	ServicesSection plus CTA; four service doors from local data; package links/forms via shared components.
/winter-arc	Event application	WinterArcContent; elapsed timer, event information, required answers, UTM and optional consents; POST /api/community.

All pages share the root layout, fixed Navbar and Footer unless a framework-level redirect/notFound prevents page rendering. Metadata is created through lib/metadata.ts for several pages and directly on others. Metadata and deployment URLs are configured through the shared metadata helpers and application settings.

6.2 Component composition

Home combines six sections: Hero renders service cards with orbit/constellation visuals; ServicesSection renders the local service catalog and links; ProcessSection presents the delivery process; WhyNovarchSection renders differentiators; BuildWithUsSection presents collaboration opportunities; CTASection links visitors to contact or services. Shared Container, Badge, Card and Button components supply layout, variants and link/button presentation.

Service detail pages resolve a known slug through local data and render ServicePageTemplate. It selects a service-specific animated hero, renders scope/delivery information and includes ServicePackagesSection. That section controls selected package/open state and passes a ServicePackage into ServicePackageModal. The modal owns form fields, submitting/error/success state and the two-stage contact/checkout flow described in Chapter 3.

Founders composes FounderJourneyTimeline, VentureProofGrid, OperatingBlueprint, FounderPrinciples and CofounderSection using local founder/project data.

HackTheFlagContent is a static promotional/challenge information page with Framer Motion, an embedded SVG QR path and HACKTHEFLAG_CHALLENGE_URL. It opens an external challenge.

6.3 Forms and visible state

ContactFormContent preselects service topics from service or category query parameters, validates a trimmed name and simple email pattern, waits for a signed token and posts a message to /api/contact. Phone and country code are combined by the BFF.

CommunityFormContent collects fullName, email, country, organization, role, interests, message and a referral string. It posts to /api/community with a token. WinterArcContent separately has both optional, initially unchecked consent controls and required application answers.

Winter Arc's live timer derives elapsed time from 2026-10-16T18:00:00+02:00, clamps negative values at zero, displays total hours plus minute/second remainders, and updates every second after mount. The display focuses on total elapsed hours with minute and second remainders. Initial zeros/mounted gating keep the server and first client render deterministic.

PaymentSuccessContent reads session_id using useSearchParams inside Suspense, asks Express to retrieve the Stripe session, then shows verification loading, success or an error. A paid Stripe session response and successful webhook persistence can occur at different times.

6.4 Shared state, styling and behavior

The global image/SVG/canvas rule constrains width and uses auto height. Responsive grids and sm/md/lg/xl utilities adjust page layouts; Navbar switches desktop navigation and a fixed mobile drawer, with body scroll locking while open.

ScrollToTop watches pathname. Hash navigation attempts to find and scroll to the target; otherwise it resets scroll immediately and on the next animation frame. useInView uses IntersectionObserver (threshold 0.1 by default, once by default); Reveal maps it to CSS opacity/transform states. useScrolled throttles passive scroll work with requestAnimationFrame. useMediaQuery uses matchMedia and useSyncExternalStore with a false server snapshot. Service graphics can be dynamically loaded with ssr:false from the client template. Framer Motion and CSS reduced-motion support should both be considered when maintaining animation accessibility.

6.5 Language switching and legal content

LanguageSwitcher loads Google's website Translate script once through a shared promise and initializes the hidden mount target. It polls for .goog-te-combo every 100 ms up to 15 seconds, then drives the select's change event. German preference is saved in novarch_lang/localStorage and googtrans cookie; legacy domain cookies are cleared on selection. English selection reloads the original React document. The active DE state follows Google's translated-ltr HTML class.

Google's supported-language request is JSONP, so translate-pa.googleapis.com must be permitted in script-src as well as connect-src. next.config.ts contains these origins and allows same-origin frames used by the widget. This machine-translation path is independent of the private /api/translate endpoint.

EventLegalHub provides confidentiality, participant-data, photography, challenge and participation sections. Its bilingual sections organize event guidance and organizer-managed legal content. PermanentLegalPage provides dedicated locations for operator and privacy information. Approved legal content and event procedures are managed by the operator.

7. Administrative dashboard

7.1 Routes and access

App initializes its authentication flag from the presence of novarch_admin_token in localStorage or sessionStorage. Without a token it returns LoginPage before the router. On login, rememberMe (default true) selects localStorage; otherwise sessionStorage. User email is stored alongside the token. Token presence is a UI gate only. Express validates all protected data/actions using the live admin session.

Dashboard route	View / action	API dependency
/	Command Center, contact metrics and all inquiries	GET /api/health, /api/dashboard/stats, /api/contacts, /api/community
/service-leads	Filtered service package inquiries	Same shared list, client category filter
/messages	Filtered general messages	Same shared list, client category filter
/inquiry/:id	Full inquiry details, copy, status, delete, mailto	Loads from list; fallback GET /api/contacts; PATCH /api/contacts/:id/status; DELETE /api/contacts/:id
/winter-arc	Winter Arc applications table and reply control	Shared GET /api/community; client applicationType filter; PATCH status; POST reply
/community	General member applications table	Shared GET /api/community; client applicationType filter; PATCH status
/winter-arc/:id and /community/:id	Application/profile details, status, delete, Winter Arc reply	Loaded member memory; PATCH /api/community/:id/status; DELETE /api/community/:id; POST /api/community/:id/reply
*	Redirect to /	No separate API

The exact inquiry detail route is source-verified in App and referenced by InquiryTable. Sidebar labels the user Super Admin, but the issued/validated backend role is admin.

7.2 Data loading and API client

loadData first reads health, then concurrently loads stats, contacts and all community applications. It updates React state only after all three calls succeed. Polling runs every 60 seconds while authenticated/visible, stops when hidden and reloads immediately on visibility return. Manual refresh uses a spinner with a minimum completion delay. The displayed dashboard clock updates every ten seconds.

services/api.ts normalizes VITE_API_URL to end in /api, attaches Bearer authorization, deduplicates simultaneous GETs by URL and removes requests from the map on settlement. Other methods mostly throw an HTTP status-text error. Logout calls the server revocation endpoint first; if it fails, App keeps the session and reports a retry message.

7.3 Tables, details and feature mapping

InquiryTable receives filtered records from DashboardView. Search matches name, email, company, planName or serviceType. Status/category filters are client-side, and useDeferredValue reduces search update pressure. Its CSV rows are joined with commas without a full quoting/formula-neutralization layer.

CommunityTable filters by status and text across member profile fields. It navigates to the type-specific detail route and can open ReplyModal for Winter Arc. ReplyModal sends trimmed subject/message to the protected backend and shows submitting, success or server-derived error state. Application deletion uses a browser confirmation and then the parent callback.

Feature	Client operation	Server enforcement / side effect
Login	api.login(email,password)	bcrypt verification; JWT+session creation; audit
List/search/filter	getContacts/getCommunityMembers; React filters	Admin validation for lists; no permission check per row
Inquiry status	updateContactStatus	Valid status enum; Contact update and audit
Application status	updateCommunityStatus	Valid normalized enum; Community update
Delete inquiry	deleteContact	Hard Contact deletion and audit
Delete applicant	deleteCommunityMember	Hard Community deletion
Reply to event applicant	replyWinterArc	Must be winter-arc; transactional email, no reply-history persistence
Withdraw marketing consent	No dashboard method/button found	Protected backend route exists; false + withdrawal timestamp
Export / copy / mailto	Browser download, clipboard or mail client	No server action or delivery confirmation
Logout	api.logout then clear browser storage	Live session deletion and audit

7.4 Dashboard workflow and session continuity

The dashboard maintains shared React state for efficient navigation between lists and detail views. Manual refresh and visibility-aware polling synchronize the displayed records with the backend. InquiryDetailPage can retrieve the contacts list when opening a record directly, while CommunityDetailPage uses the shared member collection loaded by App.

Server-side session validation protects administrative API actions independently of the browser's presentation state. The API client handles an expired session by clearing the browser token and returning the user to login. Logout coordinates server revocation with browser storage cleanup. Backend responses provide the authoritative result for persisted changes.

8. Database schema and persistence

8.1 Connection and collections

Mongoose connects using MONGODB_URI with serverSelectionTimeoutMS 8000, connectTimeoutMS 10000, socketTimeoutMS 45000, pool max 10/min 1, idle timeout 30000 and IPv4 family 4. The module sets explicit DNS resolver addresses for SRV troubleshooting. Collection names below use Mongoose's normal model-name pluralization; actual deployed names/indexes must be confirmed in the target database.

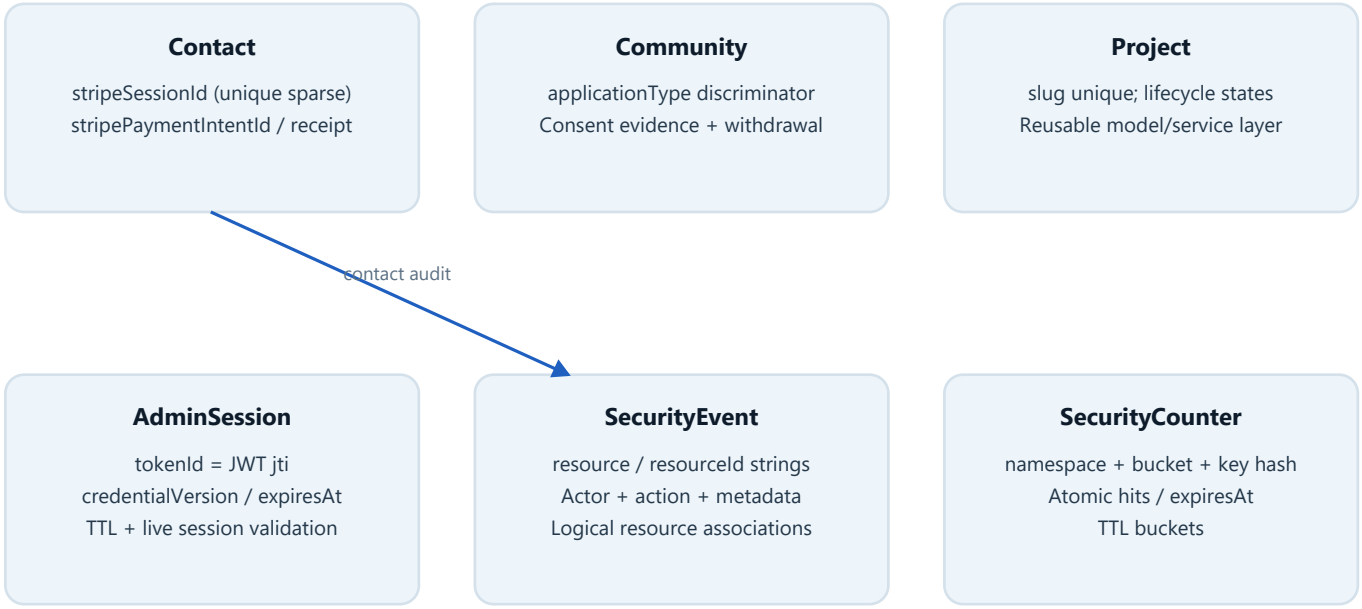


Figure 4. Logical model associations support application workflows; TTL indexes manage sessions and counters.

Model	Expected collection	Main readers / writers
Contact	contacts	Contact repository, dashboard metrics and Stripe checkout/webhook
Community	communities	Community repository, applications lists/status/withdrawal
Project	projects	Project repository/service only; no mounted HTTP routes
AdminSession	adminsessions	Auth login/logout and validateSession
SecurityEvent	securityevents	AuditService for login/logout, contacts and project service operations
SecurityCounter	securitycounters	MongoRateStore for shared general/sensitive/account/public-form buckets

Contacts, Community and Projects use timestamps and serialize _id as string id while removing _id/__v. Repository formatters normalize createdAt/updatedAt to ISO strings. Business associations use session IDs, token IDs and resource strings.

8.2 Contact fields

Fields	Storage type / default / constraint
_id; createdAt; updatedAt	ObjectId; automatic Date timestamps
name; email	Required strings, trimmed; email lowercased
submissionType	message or service_lead; default message
phone; countryCode; company	Optional trimmed strings
planName; planPrice	Optional trimmed strings
planTier	Optional Basic or Premium enum
performances; news	Array of trimmed strings; optional trimmed narrative
serviceType; preferredService	String default General Inquiry; optional string
message	Optional trimmed string
status	Seven-value lead/payment enum; default New
estimatedValue	Number; model default 490, normally derived explicitly in repository
source	Website Form, Service Package, Direct API or Referral; model default Website Form
paymentReceivedAt	Optional Date; immutable receipt convention enforced by webhook query, not schema immutable flag
stripeSessionId; stripePaymentIntentId	Optional trimmed strings; each sparse unique index

Indexes: status; submissionType; createdAt descending; email (nonunique); status+createdAt descending; submissionType+createdAt descending; sparse unique payment identifiers. Multiple submissions from one email are allowed.

8.3 Community fields

Fields	Storage type / default / constraint
_id; createdAt; updatedAt	ObjectId and automatic timestamps
applicationType	community or winter-arc; default community
fullName; email	Required strings; trim; lowercase email
country; city; contactNumber; countryCode	Optional trimmed strings
organization; role	Optional trimmed strings
whatYouDo; whatCanYouOffer; whatAreYouLookingFor	Optional trimmed strings at database level; Winter Arc API requires them
linkedinUrl; portfolioUrl; proposedGuest	Optional trimmed strings
interests; message	Trimmed string array; optional trimmed string
status	Seven community/event statuses; default New
source; referral	String default Website Form; Mixed referral value
marketingConsent; photoConsent	Boolean defaults false
marketingConsentAt; photoConsentAt	Date defaults null
marketingConsentVersion; photoConsentVersion	Optional trimmed strings
marketingConsentWithdrawnAt	Date default null

Indexes: status; applicationType; createdAt descending; email (nonunique); applicationType+status+createdAt descending; marketingConsent+applicationType. API validation is stricter than database validation for Winter Arc fields and URL safety.

8.4 Projects and security records

Model / fields	Storage rules
Project slug, name, period, description	Required strings; slug unique/trim/lowercase/indexed
Project category / status	venture, client, lab, internal or case-study (default venture); draft, published or archived (default draft)
Project summary, image, liveUrl, websiteUrl, githubUrl	Optional strings; DTOs enforce URL/relative image formats if used
Project tags / techStack	String arrays; tags defaulted by DTO
Project metrics	Subdocuments with required label/value strings
Project links	Subdocuments with required label/url; isExternal default true
Project isSample / featured	Boolean defaults false; featured indexed
Project timestamps / indexes	Automatic timestamps; status+category and status+createdAt descending plus field indexes
AdminSession tokenId	Required unique string, matches JWT jti
AdminSession username / credentialVersion	Required strings; credentialVersion is HMAC of configured password hash using JWT secret
AdminSession expiresAt	Required Date, TTL expireAfterSeconds 0; explicit expiry filter also enforced at read
SecurityEvent	actor, action, resource, resourceId, status strings; timestamp default Date.now; metadata Mixed
SecurityCounter	String _id composed of namespace, time bucket and SHA256 key; hits Number default 0; expiresAt Date with TTL 0

Counter increments are atomic findOneAndUpdate/upsert with duplicate-key retry. Session validation also compares expiresAt rather than trusting TTL cleanup timing.

8.5 Data queries and lifecycle

Contact/community lists use find().sort({createdAt:-1}).lean(); community adds applicationType only for recognized query values. Detail reads use findById. Status changes use findByIdAndUpdate with validators; deletion uses findByIdAndDelete. Consent withdrawal updates false/withdrawal/updated timestamps. Project search escapes regex metacharacters before case-insensitive name/description/tag matching; project list/status/category/featured filters are internal service capabilities only.

Repositories provide explicit record lifecycle operations, and TTL indexes manage expiring sessions and rate-limit buckets.

9. Authentication, authorization and security

9.1 Admin session flow

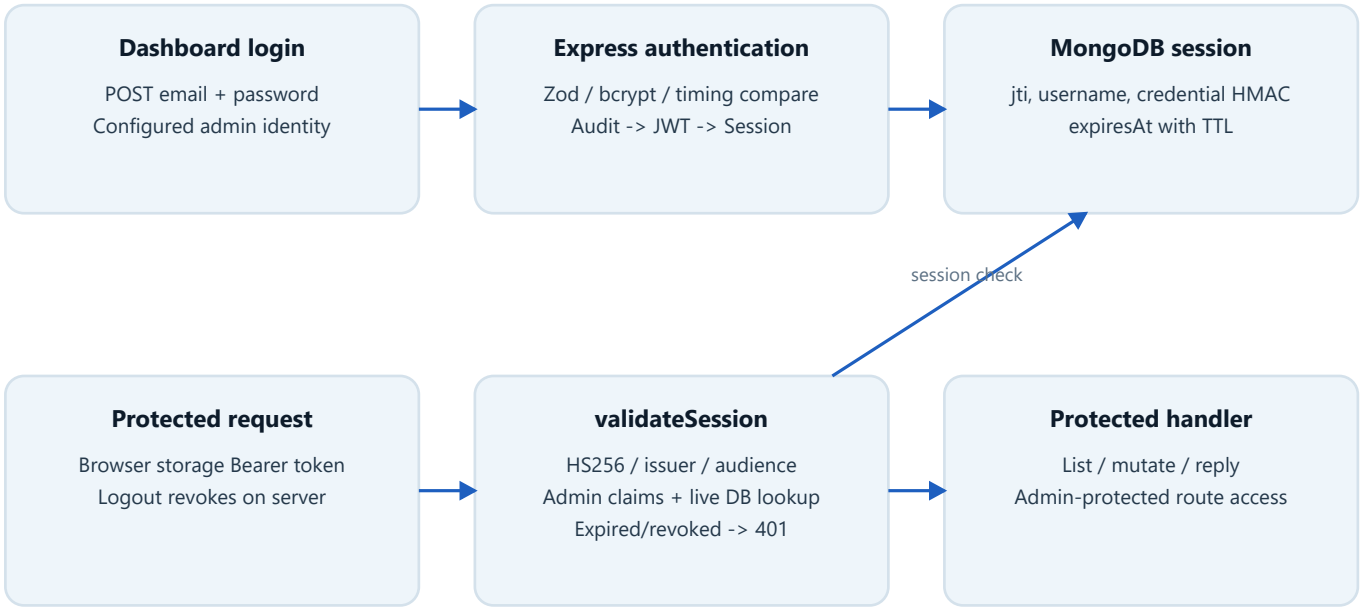


Figure 5. Two-hour admin JWTs require a live, unexpired, credential-bound MongoDB session.

Administrator login uses the environment-configured identity, Zod validation, bcrypt verification and a persisted two-hour HS256 session. JWT claims include the administrator role, issuer, audience and a unique session identifier.

validateSession pins algorithm/issuer/audience and checks admin role, configured username and string jti. It requires a matching unexpired database session with the current credentialVersion. Changing the configured password hash invalidates old sessions even if JWT time remains. Logout deletes the current session and audits the operation; password/JWT rotation and session expiry are therefore server-enforced.

Registered administrative routes use verifyAdminToken and live-session validation. Reusable permission helpers define named roles and permissions for the service layer.

9.2 Implemented security controls

Control	Verified implementation
Auth validation	bcrypt + pinned JWT + live Mongo session
Input boundaries	BFF bounded JSON/types; backend Zod DTOs; ID pattern
Internal forms	X-Internal-Token with timing-safe compare when configured
Form protection	HMAC age token, honeypot, BFF and backend limits
Rate limiting	General 200/15m, sensitive 20/15m, login-account 30/15m, public-form 8/15m
Shared counters	MongoRateStore and hashed bucket keys
CORS	Exact configured/default origin allowlist, credentials true
Headers	Express Helmet; website Next config; dashboard Vercel CSP and hardening headers
Email HTML	htmlValues/escapeHtml and unsafe URL scheme neutralization
Stripe	Raw body + signature verification + catalog price + atomic receipt guard
Secrets	Environment variables; SMTP AES-256-GCM ciphertext
Auditing	Persistent SecurityEvent for selected operations

General limiter skips both webhook aliases and authenticated BFF POSTs to /api/contact and /api/community. Sensitive limiter applies to /api/auth/login and both checkout-create aliases. Public-form limiter follows internal authentication and can use validated X-Form-Client-IP when the shared secret is set. BFF guards use an instance-local map capped around 5,000 active keys and an eight-submission rolling window. Backend buckets are shared fixed windows; list/read polling also consumes the general limit.

9.3 Production configuration and release readiness

- Configure a unique JWT signing secret and a bcrypt admin password hash with cost 12-16. Production startup validation checks the required authentication and database settings.
- Configure matching internal API secrets on the website and backend. The website's dedicated form-signing secret supports timestamped form tokens.
- Align CORS origins, public API URLs and Stripe redirect URLs with the selected deployment domains.
- Set proxy hop trust and verified client-IP forwarding to match the hosting topology so submission controls consistently identify client networks.
- Preserve raw webhook bytes and deploy the declared payment-identifier indexes to support signature verification and receipt reconciliation.
- Keep provider keys in server-side configuration. Browser-facing variables supply public routing information.
- Use the backend payment receipt marker and Stripe session details for financial reconciliation, alongside the dashboard's operational status labels.
- Apply the release checks in Chapters 11 and 12 to the intended environment. This reference is based on source inspection and document verification.

10. External integrations

Integration	Trigger / modules	Authentication and exchanged data
MongoDB	config/database; repositories; sessions; audit; counters	MONGODB_URI; contact/applicant narratives, payment references, session claims and audit metadata
Stripe Checkout	routes/checkout; ServicePackageModal	STRIPE_SECRET_KEY; customer email, name/contact/company/notes metadata and canonical package price
Stripe webhook	checkout router + raw parser	stripe-signature + STRIPE_WEBHOOK_SECRET; event/session/payment identifiers and amount
Resend	EmailService.sendEmail	RESEND_API_KEY Bearer; EMAIL_FROM, recipient, reply-to, subject, plain text/HTML
Encrypted SMTP	config/smtp; utils/crypto; EmailService	AES-GCM encrypted username/password plus key; decrypted auth to Nodemailer
Google website Translate	layout mount; LanguageSwitcher; globals.css; Next CSP	Client script/cookie; page DOM text to external widget
DeepL API	private website /api/translate	DEEPL_API_KEY header; text array, source and target
Google Cloud Translation	private website /api/translate	GOOGLE_TRANSLATE_API_KEY in provider URL; text array/source/target
Google Fonts	Next Inter font; CSS/CSP	Public font delivery
External challenge	HACKTHEFLAG_CHALLENGE_URL; page QR/link	Opens separate challenge host

Email templates include user-entered narratives and optional profile links; Resend/SMTP therefore process these fields for transactional purposes. Consent for future marketing is independent of registration acknowledgments and admin replies.

SMTP encryption format is ivHex:authTagHex:ciphertextHex with AES-256-GCM, random 12-byte IV and a 32-byte key supplied as 64 hexadecimal characters. The code supports SMTP_APP_PASSWORD_ENC or legacy SMTP_PASSWORD_ENC.

11. Environment configuration and deployment

11.1 Environment-variable reference

Values are intentionally excluded. Public prefixes are build-time browser-visible configuration and must never contain secrets.

Variable / application	Required or default	Purpose	Source
PORT (Backend)	Default 5000	Express listen port	backend/src/config/index.ts
NODE_ENV (All)	Default development in backend; set production at deploy	Production validation, logging/compiler behavior, test/listen behavior	backend/config; frontend/next.config.ts; formGuard
VERCEL (Website/backend)	Platform-injected where deployed	Suppress backend listen; choose trusted Vercel client-IP header	server.ts; lib/backend.ts; formGuard.ts
TRUST_PROXY_HOPS (Backend)	Optional; 0 default; integer 0-5	Express trusted proxy hop count for IP interpretation	backend/src/server.ts
CORS_ORIGINS (Backend)	Optional explicit comma list; defaults in config	Exact browser origins allowed; credentials true	backend/src/config/index.ts
MONGODB_URI (Backend)	Required for actual operation; enforced in production	MongoDB connection URI, never publish value	config/index.ts; config/database.ts
JWT_SECRET (Backend)	Required auth; production >=32 non-placeholder chars	HS256 signing and credential-version HMAC	config/index.ts; auth controller; session service
ADMIN_USERNAME (Backend)	Default configured email in source; set explicitly	Single normalized admin login identity	backend/src/config/index.ts
ADMIN_PASSWORD (Backend)	Required auth; bcrypt hash cost 12-16	Configured password hash, not plaintext	config/index.ts; auth controller; session service
INTERNAL_API_SECRET (Website/backend)	Required backend production >=32; optional dev	Shared BFF token; identical on both services	backend/internalApi; frontend/lib/backend
FORM_SIGNING_SECRET (Website)	Required production >=32; dev fallback exists	HMAC form timestamp tokens	frontend/src/lib/formGuard.ts
BACKEND_API_URL (Website server)	Optional; highest-priority BFF base	Express base; /api suffix normalized	frontend/src/lib/backend.ts
NEXT_PUBLIC_API_URL (Website build/browser)	Optional; hosted backend fallback	Direct checkout/verification and CSP API origin; BFF fallback	ServicePackageModal; payment-success; lib/backend; next.config
FRONTEND_URL (Backend)	Default https://novarch.eu	Stripe success/cancel redirects	backend/src/config/index.ts; checkout routes
STRIPE_SECRET_KEY (Backend)	Required checkout feature	Stripe secret API key; never browser-visible	config/index.ts; checkout routes
STRIPE_WEBHOOK_SECRET (Backend)	Required webhook processing	Signing secret for selected webhook endpoint	config/index.ts; checkout routes
RESEND_API_KEY (Backend)	Optional; email provider preferred if set	Bearer authentication to Resend	backend/src/services/email.service.ts
EMAIL_FROM (Backend)	Optional source default sender	Resend sender identity; configure verified sender	backend/src/services/email.service.ts

Variable / application	Required or default	Purpose	Source
ADMIN_NOTIFICATION_EMAIL (Backend)	Optional notification override	Preferred admin recipient for intake notifications	backend/src/services/email.service.ts
NOTIFICATION_EMAIL (Backend)	Optional legacy fallback	Admin recipient fallback after ADMIN_NOTIFICATION_EMAIL	backend/src/services/email.service.ts
EMAIL_ENCRYPTION_KEY (Backend)	Required for SMTP; 64 hex chars	32-byte AES-GCM decryption key	config/smtp.ts; utils/crypto.ts
SMTP_USERNAME_ENC (Backend)	Required for SMTP	Encrypted username, iv:tag:ciphertext format	backend/src/config/smtp.ts
SMTP_APP_PASSWORD_ENC (Backend)	Required for SMTP unless legacy alternate	Preferred encrypted app password	backend/src/config/smtp.ts
SMTP_PASSWORD_ENC (Backend)	Legacy alternate	Fallback encrypted SMTP password	backend/src/config/smtp.ts
SMTP_HOST (Backend)	Default smtp.gmail.com	SMTP server	backend/src/config/smtp.ts
SMTP_PORT (Backend)	Default 465	SMTP server port parsed as integer	backend/src/config/smtp.ts
SMTP_USE_SSL (Backend)	Default true; literal false disables	Nodemailer secure transport flag	backend/src/config/smtp.ts
SMTP_FROM_EMAIL (Backend)	Optional; decrypted username fallback	SMTP sender and notification-recipient fallback	config/smtp.ts; email service
SMTP_FROM_NAME (Backend)	Optional; default Novarch Support	SMTP display name	backend/src/config/smtp.ts
SMTP_TIMEOUT_SECONDS (Backend)	Default 30	Connection/greeting/socket timeouts	backend/src/config/smtp.ts
TRANSLATION_API_SECRET (Website server)	Required private translation route >=32	X-Translation-Token validation; route 404 if missing	frontend/src/app/api/translate/route.ts
DEEPL_API_KEY (Website server)	Optional translation provider	DeepL Auth-Key; :fx selects free endpoint	frontend/src/app/api/translate/route.ts
GOOGLE_TRANSLATE_API_KEY (Website server)	Optional translation provider	Google Cloud Translation API key	frontend/src/app/api/translate/route.ts
VITE_API_URL (Dashboard build/browser)	Optional absolute hosted fallback	Express base normalized to /api; /api uses Vite local proxy	dashboard/src/services/api.ts
VITE_FRONTEND_URL (Dashboard build/browser)	Default http://localhost:3000	Computed in MainLayout/passed as prop; frontend URL configuration	dashboard/src/App.tsx; Sidebar props
NOVARCH_SECURE_JWT_SECRET (Local helper only)	Not normal application runtime configuration	Input read by setup-local-security utility; not required by deployed services	backend/src/scripts/setup-local-security.cjs

11.2 Local setup and verified commands

Install dependencies in each app separately using the checked-in lockfile. Configure a local/test MongoDB URI and server secrets in private environment files or the deployment secret store.

```
npm --prefix frontend ci
npm --prefix backend ci
npm --prefix dashboard ci

npm run dev:backend
npm run dev:frontend
npm run dev:dashboard
```

Run the three development commands in separate sessions. Root package scripts forward into each package. The backend dev script uses Windows .\\node_modules\\.bin\\tsx watch src/server.ts syntax; on another OS invoke the installed tsx watcher with the equivalent path appropriate to that shell. Default ports are backend 5000, website 3000 and dashboard 5173. Dashboard Vite config proxies /api to localhost:5000, but its API client defaults to an absolute hosted backend; set VITE_API_URL=/api to actually use the local proxy. Set NEXT_PUBLIC_API_URL for direct browser checkout and BACKEND_API_URL for website server forwarding; keep both aligned with intended backend.

```
npm run build:frontend
npm run build:backend
npm run build:dashboard
npm run build:all

npm --prefix backend start
npm --prefix frontend start
npm --prefix dashboard run preview
npm --prefix frontend run lint
npm --prefix backend run test:security
npm --prefix backend run hash-password
```

Backend build emits dist from src with ES2022/NodeNext/strict checking. Next performs its own optimized build.

11.3 Provisioning and deployment architecture

The repository includes Vercel configuration for all three applications. Frontend declares framework nextjs. Backend uses @vercel/node with a catch-all route to src/server.ts and suppresses app.listen when VERCEL is present. Dashboard rewrites every path to index.html for BrowserRouter deep links and installs its own response headers.

Provision MongoDB with the intended database/user/network policy. Confirm sparse unique Stripe indexes, session token uniqueness and TTL indexes before relying on idempotency/expiry guarantees.

Configure identical INTERNAL_API_SECRET values on the website server and backend, a dedicated production FORM_SIGNING_SECRET on the website, admin hash/JWT/database values on Express, and public API base URLs only where browser communication requires them. CORS_ORIGINS must include the actual website and dashboard origins exactly. FRONTEND_URL controls Stripe redirects. Configure Stripe webhook delivery to one backend webhook alias with the matching webhook secret. The webhook origin/path is distinct from the website BFF origin.

Configure one email provider and approved sender/recipient settings. Verify sender authorization and transactional delivery independently.

11.4 Operations and service management

Operational area	Recommended configuration and review
Public forms	Maintain the production form-signing secret and review the form-token response during acceptance checks.
Service communication	Keep website forwarding settings, backend URLs and shared internal tokens aligned.
Submission controls	Configure trusted proxy/IP handling to support consistent rate-limit behavior.
Administrator access	Maintain the configured bcrypt hash, signing secret and live database sessions.
Payments	Review package identifiers, signed webhook delivery and payment receipt records together.
Email	Configure the preferred provider and approved sender identity, and verify transactional delivery in the target environment.
Language switching	Preserve the Google script/network origins in the website policy and include EN/DE switching in browser checks.
Dashboard data	Use shared loading, manual refresh and visibility-aware polling for synchronized record views.
Hosting	Preserve the dashboard SPA rewrite and application-specific response headers.

Server logs provide database state, record identifiers, audit action summaries and provider outcomes. Health reporting supplies database connection status and rounded process uptime. Production Next.js keeps error/warning console output while removing other console calls. These application signals support the operator's deployment monitoring and support procedures.

12. Testing and ongoing maintenance

12.1 Existing test inventory

Test source	What it covers	Execution status for this task
backend/src/scripts/security.test.ts	Escaped HTML/unsafe URLs; hostile/oversized auth input; DB fail-closed writes; JWT claims/algorithm/session revocation; unsigned webhook rejection; failure identity; paid webhook amount/idempotency with mocked model methods	Source-reviewed
frontend/src/lib/requestBody.test.ts	Null/array/malformed JSON; object-valued fields; invalid consent type; oversized request; valid consent/referral preservation	Source-reviewed
backend/src/scripts/test-consent.ts	Consent defaults/evidence, independence, withdrawal and eligibility scenarios through services	Reviewed, Source-reviewed; integration scenarios use configured services
backend/src/scripts/test-winter-arc.ts	Schema acceptance/rejection, standard community and Winter Arc service workflow/status scenarios	Reviewed, Source-reviewed; integration scenarios use configured services

test:security invokes tsx --test against the security test file. A manual example command from the root, if the installed runner is available, is node backend/node_modules/tsx/dist/cli.mjs --test frontend/src/lib/requestBody.test.ts.

12.2 Operational scripts

Script	Actual responsibility / caution
hash-password.ts	Reads one password line from stdin, enforces 12-72 UTF-8 bytes and prints a cost-12 bcrypt hash; no database update
set-admin-password.cjs	Local credential-setup helper that rewrites environment configuration; inspect before use, Source-reviewed for this documentation
setup-local-security.cjs	Local security/environment setup helper; not a production deployment/migration tool; Source-reviewed for this documentation
clear-db.ts	Connects using configured URI and calls ContactModel.deleteMany({}); scoped Contact data administration utility; Source-reviewed for this documentation
test-consent.ts / test-winter-arc.ts	Service-level integration scripts, not isolated fixtures; may have database and email side effects

12.3 Maintenance and continuous development

Area	Positive maintenance practice
Application structure	Preserve the separation between public website, API service and command portal as features evolve.
API contracts	Keep schemas, example payloads and dashboard callers aligned with registered routes.
Service catalog	Review the website presentation and authoritative backend package catalog together when updating offerings.
Payment operations	Review signed webhook delivery, receipt identifiers and provider records as part of routine reconciliation.
Administrative workflows	Include status changes, record views and refresh behavior in release acceptance checks.
Consent records	Maintain the separate consent choices, timestamps and withdrawal evidence throughout event operations.
Localization	Include English/German switching and interactive forms in browser acceptance checks.
Data operations	Maintain approved backup, restore, retention and access procedures for the deployment environment.
Secure configuration	Rotate credentials through the deployment secret store and review approved service origins regularly.
Documentation	Update the reference alongside application changes so future developers retain a reliable implementation guide.

Dependency manifests and lockfiles provide the version baseline for planned upgrades. The modular service, repository and component boundaries provide clear places to extend functionality while keeping existing contracts consistent.

13. Verification record and handover

The documentation was prepared through source review across the public website, Express backend, administrative dashboard, models, middleware, services, configuration and operational scripts. Endpoint registrations were cross-checked against router mounts, handlers, schemas and frontend callers. The page inventory includes active pages and compatibility redirects.

Examples use synthetic identifiers and example.com addresses. Credentials, private configuration values and real applicant records are excluded. API examples illustrate source-defined request and response shapes. Review scope: source analysis and PDF verification; runtime acceptance testing and provider provisioning are separate deployment activities.

All documentation deliverables reside under docs/. Preservation checks compare SHA-256 hashes of the existing application files. PDF verification covers page rendering, readable text, internal navigation, tables, diagrams and page bounds. The resulting reference supports client handover, onboarding, operations and future development.

14. File and module reference

Each entry below identifies responsibility plus exported symbols where useful.

frontend/src/app

File / lines	Responsibility and key surface
frontend/src/app/about/page.tsx (53)	App Router page/redirect; exact route and behavior in Chapter 6. Exports: AboutPage, metadata.
frontend/src/app/api/community/route.ts (135)	Website server endpoint; full contract and origin distinction in Chapter 5. Exports: POST.
frontend/src/app/api/contact/route.ts (101)	Website server endpoint; full contract and origin distinction in Chapter 5. Exports: POST.
frontend/src/app/api/form-token/route.ts (17)	Website server endpoint; full contract and origin distinction in Chapter 5. Exports: GET.
frontend/src/app/api/translate/route.ts (103)	Website server endpoint; full contract and origin distinction in Chapter 5. Exports: POST.
frontend/src/app/api/winter-arc/route.ts (5)	Website server endpoint; full contract and origin distinction in Chapter 5. Exports: POST.
frontend/src/app/case-studies/[slug]/page.tsx (5)	App Router page/redirect; exact route and behavior in Chapter 6. Exports: CaseStudySlugRedirect.
frontend/src/app/case-studies/page.tsx (5)	App Router page/redirect; exact route and behavior in Chapter 6. Exports: CaseStudiesRedirect.
frontend/src/app/contact/page.tsx (442)	App Router page/redirect; exact route and behavior in Chapter 6. Exports: ContactPage.
frontend/src/app/datenschutz/page.tsx (4)	App Router page/redirect; exact route and behavior in Chapter 6. Exports: metadata, Page.
frontend/src/app/founders/page.tsx (260)	App Router page/redirect; exact route and behavior in Chapter 6. Exports: metadata, FoundersPage.
frontend/src/app/globals.css (288)	Global Tailwind tokens, reset, accessibility, animation and Translate override rules.
frontend/src/app/hacktheflag/page.tsx (46)	App Router page/redirect; exact route and behavior in Chapter 6. Exports: metadata, HackTheFlagPage.
frontend/src/app/impressum/page.tsx (4)	App Router page/redirect; exact route and behavior in Chapter 6. Exports: metadata, Page.
frontend/src/app/layout.tsx (58)	Shared server layout: dark theme, font, navbar, main and footer. Exports: metadata, RootLayout.
frontend/src/app/legal/nda/page.tsx (5)	App Router page/redirect; exact route and behavior in Chapter 6. Exports: Page.
frontend/src/app/legal/page.tsx (5)	App Router page/redirect; exact route and behavior in Chapter 6. Exports: Page.
frontend/src/app/legal/photography-consent/page.tsx (5)	App Router page/redirect; exact route and behavior in Chapter 6. Exports: Page.
frontend/src/app/legal/winter-arc-2026/page.tsx (28)	App Router page/redirect; exact route and behavior in Chapter 6. Exports: metadata, WinterArcLegalPage.
frontend/src/app/our-community/page.tsx (560)	App Router page/redirect; exact route and behavior in Chapter 6. Exports: OurCommunityPage.
frontend/src/app/page.tsx (37)	App Router page/redirect; exact route and behavior in Chapter 6. Exports: metadata, HomePage.
frontend/src/app/payment-success/page.tsx (149)	App Router page/redirect; exact route and behavior in Chapter 6. Exports: PaymentSuccessPage.
frontend/src/app/pricing/page.tsx (44)	App Router page/redirect; exact route and behavior in Chapter 6. Exports: metadata, PricingPage.

File / lines	Responsibility and key surface
frontend/src/app/services/ai-automation/page.tsx (5)	App Router page/redirect; exact route and behavior in Chapter 6. Exports: AiAutomationRedirect.
frontend/src/app/services/ai-workflow/page.tsx (20)	App Router page/redirect; exact route and behavior in Chapter 6. Exports: metadata, AiWorkflowPage.
frontend/src/app/services/ai-workflows/page.tsx (5)	App Router page/redirect; exact route and behavior in Chapter 6. Exports: AiWorkflowsRedirect.
frontend/src/app/services/automation-integration/page.tsx (20)	App Router page/redirect; exact route and behavior in Chapter 6. Exports: metadata, AutomationIntegrationPage.
frontend/src/app/services/custom-software/page.tsx (20)	App Router page/redirect; exact route and behavior in Chapter 6. Exports: metadata, CustomSoftwarePage.
frontend/src/app/services/digital-launch/page.tsx (20)	App Router page/redirect; exact route and behavior in Chapter 6. Exports: metadata, DigitalLaunchPage.
frontend/src/app/services/page.tsx (35)	App Router page/redirect; exact route and behavior in Chapter 6. Exports: metadata, ServicesOverviewPage.
frontend/src/app/winter-arc/page.tsx (46)	App Router page/redirect; exact route and behavior in Chapter 6. Exports: metadata, WinterArcPage.

frontend/src/components

File / lines	Responsibility and key surface
frontend/src/components/animations/AIWorkflowHeroGraphic.tsx (229)	Presentational animation/graphic; supports section/page rendering, not business API processing. Exports: AIWorkflowHeroGraphic.
frontend/src/components/animations/AutomationIntegrationAnimation.tsx (722)	Presentational animation/graphic; supports section/page rendering, not business API processing. Exports: AutomationIntegrationAnimation.
frontend/src/components/animations/ConstellationBackground.tsx (159)	Presentational animation/graphic; supports section/page rendering, not business API processing. Exports: ConstellationBackground.
frontend/src/components/animations/CustomSoftwareAnimation.tsx (573)	Presentational animation/graphic; supports section/page rendering, not business API processing. Exports: CustomSoftwareAnimation.
frontend/src/components/animations/DigitalLaunchFlowAnimation.tsx (324)	Presentational animation/graphic; supports section/page rendering, not business API processing. Exports: DigitalLaunchFlowAnimation.
frontend/src/components/animations/DNAanimation.tsx (264)	Presentational animation/graphic; supports section/page rendering, not business API processing. Exports: DNAanimation.
frontend/src/components/animations/OperationalProblemSolutionAnimation.tsx (374)	Presentational animation/graphic; supports section/page rendering, not business API processing. Exports: OperationalProblemSolutionAnimation.
frontend/src/components/animations/ProcessNode.tsx (73)	Presentational animation/graphic; supports section/page rendering, not business API processing. Exports: ProcessNode.
frontend/src/components/animations/Reveal.tsx (35)	Presentational animation/graphic; supports section/page rendering, not business API processing. Exports: Reveal.
frontend/src/components/animations/ServicesOrbitAnimation.tsx (386)	Presentational animation/graphic; supports section/page rendering, not business API processing. Exports: ServicesOrbitAnimation.
frontend/src/components/layout/Footer.tsx (168)	Brand, service/company/legal links and copyright bar. Exports: Footer.
frontend/src/components/layout/LanguageSwitcher.tsx (231)	Shared Google widget readiness, preference, retry and EN/DE UI. Exports: LanguageSwitcher.
frontend/src/components/layout/MobileMenu.tsx (217)	Responsive drawer/backdrop and expanded service navigation. Exports: MobileMenu.
frontend/src/components/layout/Navbar.tsx (269)	Fixed header, service menu, language control and mobile open state. Exports: Navbar.
frontend/src/components/layout/ScrollToTop.tsx (35)	Path/hash navigation scroll management after mount. Exports: ScrollToTop.
frontend/src/components/sections/BuildWithUs.tsx (291)	Local collaboration opportunities and contact-oriented links. Exports: BuildWithUsSection.
frontend/src/components/sections/CofounderSection.tsx (172)	Local cofounder profile display with reveal/in-view presentation. Exports: CofounderSection.
frontend/src/components/sections/CTA.tsx (66)	Closing project/contact call to action and brand principles. Exports: CTASection.
frontend/src/components/sections/EventLegalHub.tsx (78)	Bilingual static legal sections/anchors with organizer-managed information. Exports: EventLegalHub.
frontend/src/components/sections/FounderJourneyTimeline.tsx (239)	Founder journey timeline with scroll-linked progress. Exports: FounderJourneyTimeline.
frontend/src/components/sections/FounderPrinciples.tsx (178)	Typed founder operating-principle cards and call to action. Exports: FounderPrinciples.

File / lines	Responsibility and key surface
frontend/src/components/sections/HackTheFlagContent.tsx (1084)	Animated external-challenge introduction and embedded QR. Exports: HackTheFlagContent.
frontend/src/components/sections/Hero.tsx (120)	Home hero and service entry cards with orbit/constellation visuals. Exports: Hero.
frontend/src/components/sections/OperatingBlueprint.tsx (115)	Typed operating steps and delivery sequence presentation. Exports: OperatingBlueprint.
frontend/src/components/sections/PermanentLegalPage.tsx (16)	Operator/privacy placeholders explicitly awaiting approved content. Exports: PermanentLegalPage.
frontend/src/components/sections/Process.tsx (293)	Delivery process presentation using local steps and node visuals. Exports: ProcessSection.
frontend/src/components/sections/ServicePackagesSection.tsx (312)	Package comparison, selection/open modal state and implementation scope UI. Exports: ServicePackagesSection.
frontend/src/components/sections/ServicePageTemplate.tsx (374)	Common detail-page composition with service-specific dynamic graphics. Exports: ServicePageTemplate.
frontend/src/components/sections/Services.tsx (166)	Local service catalog cards linking to service pages. Exports: ServicesSection.
frontend/src/components/sections/VentureProofGrid.tsx (169)	Typed venture/project proof cards and external links. Exports: VentureProofGrid.
frontend/src/components/sections/WhyNovarch.tsx (248)	Differentiator presentation from local data. Exports: WhyNovarchSection.
frontend/src/components/sections/WinterArcContent.tsx (1309)	Elapsed timer, event information, UTM and signed application/consent form. Exports: WinterArcContent.
frontend/src/components/ui/Badge.tsx (30)	Reusable semantic visual badge variants. Exports: Badge.
frontend/src/components/ui/Button.tsx (125)	Styled button/link variants and interaction props. Exports: Button.
frontend/src/components/ui/Card.tsx (29)	Reusable bordered/surface card presentation. Exports: Card.
frontend/src/components/ui/ChallengeQRCode.tsx (122)	Reusable QR presentation, clipboard URL and static QR-image download. Exports: ChallengeQRCode.
frontend/src/components/ui/Container.tsx (29)	Reusable constrained-width content and responsive padding. Exports: Container.
frontend/src/components/ui/ServicePackageModal.tsx (369)	Token-protected lead capture then direct basic-plan Stripe checkout. Exports: ServicePackageModal.

frontend/src/context

File / lines	Responsibility and key surface
frontend/src/context/ThemeContext.tsx (34)	Fixed dark theme context and no-op toggle. Exports: ThemeProvider, useTheme.

frontend/src/data

File / lines	Responsibility and key surface
frontend/src/data/buildWithUs.ts (134)	Typed application content and initial-store definitions. Exports: buildWithUsData.
frontend/src/data/contact.ts (63)	Typed application content and initial-store definitions. Exports: PerformanceOptionItem, CountryCodeItem, PERFORMANCE_OPTIONS, COUNTRY_CODES.
frontend/src/data/founders.ts (183)	Typed application content and initial-store definitions. Exports: OPPOSITE_CONTEXT, FOUNDER, COFOUNDER, ORIGIN_STORY, JOURNEY, VENTURE_PROOF, OPERATING_SEQUENCE, FOUNDER_PRINCIPLES, FOUNDER_PROMISE.
frontend/src/data/hero.ts (48)	Typed application content and initial-store definitions. Exports: HERO_SERVICES_STRIP.
frontend/src/data/navigation.ts (44)	Typed application content and initial-store definitions. Exports: navigation.
frontend/src/data/process.ts (74)	Typed application content and initial-store definitions. Exports: PROCESS_STEPS.
frontend/src/data/projects.ts (106)	Typed application content and initial-store definitions. Exports: projects, labEntries, getProjectBySlug, getLabEntryBySlug.
frontend/src/data/services.ts (519)	Typed application content and initial-store definitions. Exports: services, getServiceBySlug.
frontend/src/data/whyNovarch.ts (64)	Typed application content and initial-store definitions. Exports: DIFFERENTIATORS.

frontend/src/hooks

File / lines	Responsibility and key surface
frontend/src/hooks/useFormToken.ts (24)	One-time fetch of a form token with unmount cancellation. Exports: useFormToken.
frontend/src/hooks/useInView.ts (41)	IntersectionObserver hook, configurable once/threshold/margin. Exports: useInView.
frontend/src/hooks/useMediaQuery.ts (32)	SSR-compatible breakpoint subscription with matchMedia. Exports: useMediaQuery.
frontend/src/hooks/useScrolled.ts (32)	Passive scroll state throttled with requestAnimationFrame. Exports: useScrolled.

frontend/src/lib

File / lines	Responsibility and key surface
frontend/src/lib/backend.ts (23)	BFF backend URL normalization and internal/client-IP headers. Exports: BACKEND_API_URL, backendForwardHeaders.
frontend/src/lib/constants.ts (44)	Site identity/principles and challenge URL constants. Exports: SITE, VAT_CONFIG, ROUTES, HACKTHEFLAG_CHALLENGE_URL, BRAND_PRINCIPLES.
frontend/src/lib/formGuard.ts (100)	HMAC timestamp tokens, honeypot and process-local submission limit. Exports: issueFormToken, verifyFormToken, guardPublicForm, BACKEND_UNAVAILABLE_MESSAGE.
frontend/src/lib/metadata.ts (40)	Shared SEO/OpenGraph/Twitter metadata construction. Exports: generateMetadata.
frontend/src/lib/requestBody.test.ts (17)	Node test cases for bounded/typed request input.
frontend/src/lib/requestBody.ts (41)	Bounded streaming JSON/type boundary for website handlers. Exports: RequestBodyError, readObject.
frontend/src/lib/utils.ts (30)	Class-name helper composing clsx and tailwind merge. Exports: cn, formatPrice, clamp.

frontend/src/services

File / lines	Responsibility and key surface
frontend/src/services/content.service.ts (137)	Local typed content facade with getters and filtering. Exports: ContentService, contentService.

frontend/src/types

File / lines	Responsibility and key surface
frontend/src/types/buildWithUs.ts (33)	Typed contracts shared by presentation and data workflows. Exports: CollaborationArea, EcosystemPillar, BuildWithUsData.
frontend/src/types/case-study.ts (11)	Typed contracts shared by presentation and data workflows. Exports: CaseStudy, CaseStudyDetail.
frontend/src/types/form.ts (21)	Typed contracts shared by presentation and data workflows. Exports: PerformanceOption, ContactFormData.
frontend/src/types/founder.ts (40)	Typed contracts shared by presentation and data workflows. Exports: JourneyOppositeContext, FounderJourneyStep, VentureProof, FounderPrinciple, OperatingStep.
frontend/src/types/hero.ts (11)	Typed contracts shared by presentation and data workflows. Exports: HeroServiceItem.
frontend/src/types/navigation.ts (15)	Typed contracts shared by presentation and data workflows. Exports: NavChild, NavItem.
frontend/src/types/process.ts (13)	Typed contracts shared by presentation and data workflows. Exports: ProcessStep.
frontend/src/types/project.ts (52)	Typed contracts shared by presentation and data workflows. Exports: ProjectStatus, ProjectCategory, ProjectMetric, ProjectLink, Project, LabEntry.
frontend/src/types/service.ts (48)	Typed contracts shared by presentation and data workflows. Exports: ServiceDeliverable, ServiceScope, ServicePackage, Service.
frontend/src/types/testimonial.ts (0)	Typed contracts shared by presentation and data workflows.
frontend/src/types/whyNovarch.ts (9)	Typed contracts shared by presentation and data workflows. Exports: Differentiator.

backend/src/config

File / lines	Responsibility and key surface
backend/src/config/database.ts (53)	Runtime configuration helper; relationships and environment inputs in Chapters 3, 10 and 11. Exports: connectDatabase, isDatabaseConnected.
backend/src/config/index.ts (52)	Runtime configuration helper; relationships and environment inputs in Chapters 3, 10 and 11. Exports: config.
backend/src/config/packageCatalog.ts (64)	Runtime configuration helper; relationships and environment inputs in Chapters 3, 10 and 11. Exports: PackageCatalogItem, PACKAGE_CATALOG, getPackageBySlugOrId.
backend/src/config/smtp.ts (43)	Runtime configuration helper; relationships and environment inputs in Chapters 3, 10 and 11. Exports: SmtpConfig, getSmtpConfig.

backend/src/controllers

File / lines	Responsibility and key surface
backend/src/controllers/auth.controller.ts (52)	HTTP validation/response orchestration; detailed endpoint contract in Chapter 5. Exports: authController.
backend/src/controllers/community.controller.ts (219)	HTTP validation/response orchestration; detailed endpoint contract in Chapter 5. Exports: communityController.
backend/src/controllers/contact.controller.ts (130)	HTTP validation/response orchestration; detailed endpoint contract in Chapter 5. Exports: contactController.
backend/src/controllers/dashboard.controller.ts (16)	HTTP validation/response orchestration; detailed endpoint contract in Chapter 5. Exports: dashboardController.
backend/src/controllers/health.controller.ts (18)	HTTP validation/response orchestration; detailed endpoint contract in Chapter 5. Exports: healthController.

backend/src/data

File / lines	Responsibility and key surface
backend/src/data/store.ts (4)	Typed application content and initial-store definitions. Exports: initialInquiries.

backend/src/middleware

File / lines	Responsibility and key surface
backend/src/middleware/error.middleware.ts (29)	Request security/authorization/rate/error processing; actual mounting and limits in Chapters 2 and 9. Exports: errorHandler.
backend/src/middleware/internalApi.middleware.ts (26)	Request security/authorization/rate/error processing; actual mounting and limits in Chapters 2 and 9. Exports: requireInternalApiSecret.
backend/src/middleware/jwt.middleware.ts (40)	Request security/authorization/rate/error processing; actual mounting and limits in Chapters 2 and 9. Exports: AdminAuthRequest, verifyAdminToken.
backend/src/middleware/mongoRateStore.ts (32)	Request security/authorization/rate/error processing; actual mounting and limits in Chapters 2 and 9. Exports: MongoRateStore.
backend/src/middleware/publicFormLimiter.ts (22)	Request security/authorization/rate/error processing; actual mounting and limits in Chapters 2 and 9. Exports: publicFormLimiter.
backend/src/middleware/rbac.middleware.ts (100)	Request security/authorization/rate/error processing; actual mounting and limits in Chapters 2 and 9. Exports: Permission, Role, ROLE_PERMISSIONS, hasPermission, requirePermission.

backend/src/models

File / lines	Responsibility and key surface
backend/src/models/community.model.ts (73)	Mongoose schema/index declaration; complete field reference in Chapter 8. Exports: CommunityDocument, CommunityModel.
backend/src/models/contact.model.ts (64)	Mongoose schema/index declaration; complete field reference in Chapter 8. Exports: ContactDocument, ContactModel.
backend/src/models/project.model.ts (66)	Mongoose schema/index declaration; complete field reference in Chapter 8. Exports: ProjectDocument, ProjectModel.
backend/src/models/securityEvent.model.ts (7)	Mongoose schema/index declaration; complete field reference in Chapter 8. Exports: SecurityEventModel.
backend/src/models/session.model.ts (9)	Mongoose schema/index declaration; complete field reference in Chapter 8. Exports: SessionModel.

backend/src/repositories

File / lines	Responsibility and key surface
backend/src/repositories/community.repository.ts (271)	Model queries, CRUD and document-to-DTO normalization; persistence behavior in Chapter 8. Exports: ICommunityRepository, MongoCommunityRepository, communityRepository.
backend/src/repositories/contact.repository.ts (264)	Model queries, CRUD and document-to-DTO normalization; persistence behavior in Chapter 8. Exports: IContactRepository, MongoContactRepository, contactRepository.
backend/src/repositories/project.repository.ts (255)	Model queries, CRUD and document-to-DTO normalization; persistence behavior in Chapter 8. Exports: IProjectRepository, MongoProjectRepository, projectRepository.

backend/src/routes

File / lines	Responsibility and key surface
backend/src/routes/auth.routes.ts (10)	Express method/path registration; expanded aliases in Chapter 5.
backend/src/routes/checkout.routes.ts (260)	Express method/path registration; contains Stripe handler implementation.
backend/src/routes/community.routes.ts (20)	Express method/path registration; expanded aliases in Chapter 5.
backend/src/routes/contact.routes.ts (18)	Express method/path registration; expanded aliases in Chapter 5.
backend/src/routes/dashboard.routes.ts (10)	Express method/path registration; expanded aliases in Chapter 5.
backend/src/routes/health.routes.ts (8)	Express method/path registration; expanded aliases in Chapter 5.

backend/src/scripts

File / lines	Responsibility and key surface
backend/src/scripts/clear-db.ts (29)	Contact data administration utility; reviewed at source level.
backend/src/scripts/hash-password.ts (11)	Read stdin password and produce cost-12 bcrypt hash.
backend/src/scripts/security.test.ts (128)	Isolated security tests with invalid DB setup and mocked payment/session methods.
backend/src/scripts/test-consent.ts (209)	Service integration scenarios that can persist records/send emails.
backend/src/scripts/test-winter-arc.ts (122)	Community/Winter schema and service integration scenarios.

backend/src

File / lines	Responsibility and key surface
backend/src/server.ts (168)	Express initialization, aliases, middleware and startup/export.

backend/src/services

File / lines	Responsibility and key surface
backend/src/services/audit.service.ts (58)	Reusable business orchestration; service relationships described in Chapters 3 and 9. Exports: AuditAction, AuditStatus, AuditLogEntry, AuditService, auditService.
backend/src/services/community.service.ts (58)	Reusable business orchestration; service relationships described in Chapters 3 and 9. Exports: CommunityService, communityService.
backend/src/services/contact.service.ts (76)	Reusable business orchestration; service relationships described in Chapters 3 and 9. Exports: ContactService, contactService.
backend/src/services/dashboard.service.ts (43)	Reusable business orchestration; service relationships described in Chapters 3 and 9. Exports: DashboardService, dashboardService.
backend/src/services/email.service.ts (524)	Reusable business orchestration; service relationships described in Chapters 3 and 9. Exports: SendAcknowledgmentParams, SendEmailOptions, EmailSendResult, getAdminNotificationEmail, EmailService, emailService.
backend/src/services/project.service.ts (86)	Reusable business orchestration; service relationships described in Chapters 3 and 9. Exports: ProjectService, projectService.
backend/src/services/session.service.ts (17)	Reusable business orchestration; service relationships described in Chapters 3 and 9. Exports: credentialVersion, validateSession.

backend/src/types

File / lines	Responsibility and key surface
backend/src/types/community.types.ts (185)	TypeScript data contract and Zod runtime DTO definitions. Exports: ApplicationType, CommunityStatus, ApplicationReferral, CommunityMember, ReferralSchema, CreatePublicCommunitySchema, CreateWinterArcPublicSchema, CreateUnifiedCommunitySchema, CreatePublicCommunityDTO; +5 additional declarations.
backend/src/types/contact.types.ts (68)	TypeScript data contract and Zod runtime DTO definitions. Exports: LeadStatus, SubmissionType, ContactInquiry, CreatePublicContactSchema, CreatePublicContactDTO, CreateContactDTO, UpdateContactStatusSchema.
backend/src/types/dashboard.types.ts (16)	TypeScript data contract and Zod runtime DTO definitions. Exports: DashboardStats.
backend/src/types/project.types.ts (71)	TypeScript data contract and Zod runtime DTO definitions. Exports: ProjectStatus, ProjectCategory, ProjectMetric, ProjectLink, Project, CreateProjectSchema, UpdateProjectSchema, ProjectFilterSchema, CreateProjectDTO; +2 additional declarations.

backend/src/utils

File / lines	Responsibility and key surface
backend/src/utils/crypto.ts (43)	Resource identifier, cryptographic or email HTML helper; technical rules in Chapters 4 and 10. Exports: encryptSecret, decryptSecret, timingSafeEqualString.
backend/src/utils/html.ts (19)	Resource identifier, cryptographic or email HTML helper; technical rules in Chapters 4 and 10. Exports: escapeHtml, htmlValues.
backend/src/utils/ids.ts (5)	Resource identifier, cryptographic or email HTML helper; technical rules in Chapters 4 and 10. Exports: isValidResourceId.

dashboard/src

File / lines	Responsibility and key surface
dashboard/src/App.tsx (788)	Authentication gate, shared data/mutations, polling and all dashboard routes. Exports: App.
dashboard/src/index.css (47)	Dashboard Tailwind import, dark tokens and shared visual styles.
dashboard/src/main.tsx (10)	React StrictMode root mount and stylesheet import.
dashboard/src/vite-env.d.ts (1)	Framework/tooling type declarations.

dashboard/src/components

File / lines	Responsibility and key surface
dashboard/src/components/CommunityDetailPage.tsx (560)	Loaded-member detail view, consent context, status/delete and reply modal. Exports: CommunityDetailPage.
dashboard/src/components/CommunityTable.tsx (451)	Member/event list search/filter, status/detail navigation and Winter Arc replies. Exports: CommunityTableProps, CommunityTable.
dashboard/src/components/InquiryDetailPage.tsx (706)	Lead details, list fallback fetch, clipboard/mailto and status/delete actions. Exports: InquiryDetailPage.
dashboard/src/components/InquiryDrawer.tsx (318)	Reusable contact detail drawer with mailto/status/delete; App uses full-page details. Exports: InquiryDrawer.
dashboard/src/components/InquiryTable.tsx (417)	Contact list UI, detail navigation, status controls and client CSV download. Exports: InquiryTable.
dashboard/src/components/LoginPage.tsx (210)	Admin credential form and remember-session storage selection. Exports: LoginPage.
dashboard/src/components/ReplyModal.tsx (126)	Transactional Winter Arc reply editor with server-derived feedback. Exports: ReplyModal.
dashboard/src/components/Sidebar.tsx (315)	Command portal navigation, counts and logout control; Super Admin label is UI text. Exports: Sidebar.
dashboard/src/components/StatsCard.tsx (148)	Visual metric card component. Exports: StatsCard.
dashboard/src/components/StatusBadge.tsx (113)	Lead/community status color and label presentation. Exports: StatusBadge.

dashboard/src/services

File / lines	Responsibility and key surface
dashboard/src/services/api.ts (154)	Dashboard Bearer client, concurrent GET deduplication and 401 logout behavior. Exports: getAdminToken, api.

dashboard/src/types

File / lines	Responsibility and key surface
dashboard/src/types/index.ts (97)	Typed contracts shared by presentation and data workflows. Exports: LeadStatus, SubmissionType, CommunityStatus, ApplicationType, ApplicationReferral, ContactInquiry, CommunityMember, DashboardStats, HealthResponse.

Supporting configuration and assets

File / directory	Role
package.json	Root service/build forwarding commands for the application packages.
frontend/package.json; backend/package.json; dashboard/package.json	Application scripts and dependency manifest ranges; version authority cross-checked with each lockfile.
frontend/package-lock.json; backend/package-lock.json; dashboard/package-lock.json	npm lockfile v3 reproducibility and exact resolved versions.
frontend/next.config.ts	Website production compiler/compression, import optimization and response CSP/security/cache headers.
frontend/postcss.config.mjs; frontend/eslint.config.mjs	Tailwind PostCSS and ESLint/Next lint tooling.
frontend/tsconfig.json; backend/tsconfig.json; dashboard/tsconfig.json	Strict TypeScript configurations, path aliases and output/bundler modes.
dashboard/vite.config.ts	React/Tailwind plugins, local 5173 port, /api proxy and manual vendor chunks.
frontend/vercel.json; backend/vercel.json; dashboard/vercel.json	Separate hosting entry/rewrite/header configuration, described in Chapter 11.
*/.env.example	Configuration name templates only; actual secret values were neither copied nor documented.
backend/src/scripts/*.cjs	Local setup helpers; mutation side effects reviewed, no execution for this task.
frontend/public; dashboard/public	Static images, branding and QR assets. These are served files, not database collections.
frontend/AGENTS.md; frontend/CLAUDE.md	Repository guidance for agents.
README.md; SECURITY_REMEDIATION.md	Existing explanatory/review material; source takes precedence over claims in these documents.
.gitignore	Ignored file/build/dependency conventions; preserved unchanged.